In [1]:
```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from scipy import stats
%config InlineBackend.figure_format = 'retina'
pd.set_option('display.max_columns', None)
```

Статистика для анализа киберинцидентов: корреляционный анализ и проверка гипотез о связях

1. Понятие и классификация корреляций

В предыдущей лекции каждый признак изучался по отдельности. Мы строили их распределения и вычисляли статистики.

Но в реальном сетевом трафике и поведении пользователей признаки никогда не живут изолированно. Атака — это всегда комплексное событие, которое одновременно влияет на несколько метрик. Чтобы найти скрытые угрозы, нужно научиться видеть связи между признаками. Именно для этого в кибербезопасности применяется корреляционный анализ.

Важно отметить, что в кибербезопасности под «корреляцией» понимают два разных, но дополняющих друг друга подхода:

1. Логическая и временная корреляция (Event Correlation) — поиск связей между различными событиями безопасности во времени и пространстве (например, «если произошло событие А, а через 5 минут событие Б на другом сервере, то это атака»).
2. Статистическая корреляция (Statistical Correlation) — использование математических методов и машинного обучения для поиска взаимосвязей между переменными (например, выявление аномалий в сетевом трафике или поведении пользователей).

Различие логического подход и статистического (корреляционный).

- Логический подход: «ЕСЛИ объем исходящего трафика > 5 ГБ за час, ТОГДА выдать тревогу». Минус: много ложных срабатываний, например, когда разработчик легально загружает большой объем данных в облако.
- Статистический подход: Мы ищем не просто пороговые значения, а взаимосвязь (корреляцию) между двумя или более переменными.

Традиционные (правило-ориентированные) системы защиты эффективны против известных угроз, но бессильны перед новыми, сложными или скрытыми атаками.

Статистический подход закрывает этот критический пробел, и его применение обосновано тремя ключевыми факторами:

- Обнаружение неизвестных угроз.
- Адаптивность и контекстная точность: статистические модели автоматически формируют динамический «базовый профиль» (baseline) для каждого пользователя, устройства или сегмента сети. То, что является аномалией для менеджера, может быть абсолютной нормой для системного администратора. Это резко снижает уровень ложных срабатываний по сравнению со статическими порогами.
- Выявление скрытых взаимосвязей в больших данных: человеческий мозг и простые правила не способны отследить слабые, разрозненные сигналы в миллионах событий. Статистическая корреляция математически доказывает наличие связи между событиями (например, между временем доступа к файлу и микро-всплеском сетевого трафика), превращая «информационный шум» в четкий индикатор компрометации.

Статистическая корреляция — это мера взаимной зависимости между двумя или более переменными. По сути, она показывает, насколько изменение одной переменной связано с изменением другой. Когда значения одной переменной систематически изменяются вместе со значениями другой, можно говорить о наличии корреляционной связи.

В отличие от функциональной зависимости (где каждому значению X соответствует ровно одно значение Y, как в формуле $y = 2x + 1$), корреляция описывает стохастическую (вероятностную) связь: при изменении одной величины другая изменяется **в среднем**, но не строго детерминировано.

Простая аналогия из кибербезопасности: чем больше запросов в минуту (`requests_per_min`) приходит с IP-адреса, тем выше загрузка CPU (`cpu_load_percent`). Но это не жесткая формула: при 100 запросах CPU может быть загружен и на 60% и на 75% в зависимости от типа запросов. Это и есть корреляция, а не функциональная зависимость.

Важно отметить, что корреляция оценивает только силу и направление взаимосвязи, но не устанавливает причинно-следственных отношений между переменными.

В кибербезопасности критически важно понимать это ограничение.

- Пример: мы видим высокую корреляцию между признаками " `cpu_load_percent` " (загрузкой ЦП) и " `response_time_ms` " (временем отклика).
- Возможная ошибка аналитика: «высокая загрузка ЦП вызывает задержки отклика».
- Реальность: оба этих признака могут расти одновременно из-за третьего фактора — например, легитимного бэкапа баз данных или, наоборот, DDoS-атаки.

Корреляция в кибербезопасности — это не ответ на вопрос «почему?», а сигнал для расследования, указывающий на то, где нужно проводить более глубокий анализ.

Корреляции классифицируются по нескольким основаниям. Понимание этих различий критически важно для правильного выбора метода анализа в кибербезопасности.

По направлению связи

Тип	Описание	Пример в кибербезопасности
Положительная ($r > 0$)	Рост X сопровождается ростом Y	Чем больше `payload_size_bytes`, тем выше `response_time_ms`
Отрицательная ($r < 0$)	Рост X сопровождается падением Y	Чем выше `cpu_load_percent`, тем ниже пропускная способность сети
Отсутствие связи ($r \approx 0$)	Изменения X не влияют на Y	`user_role` и время суток запроса (обычно независимы)

По форме связи

Тип	Описание	Метод анализа
Линейная	Связь описывается прямой линией: $Y = aX + b$	Коэффициент Пирсона (r)
Нелинейная (монотонная)	Связь монотонна, но не линейна (логарифмическая, степенная и т. п.)	Коэффициенты Спирмена (ρ), Кендалла (τ)
Нелинейная (немонотонная)	Связь имеет экстремумы (U-образная, периодическая и т. п.)	Ранговые методы + визуальный анализ

Важно для кибербезопасности: сетевой трафик часто имеет нелинейные монотонные связи. Например, зависимость между количеством запросов и временем отклика экспоненциальная: до определенного порога отклик растет медленно, после — лавинообразно. Применение только линейного коэффициента Пирсона здесь даст заниженную оценку связи.

По силе связи (для коэффициента Пирсона)

Значение |r|	Сила связи	Интерпретация в ИБ
0.00 – 0.20	Очень слабая / отсутствует	Признаки практически независимы
0.20 – 0.40	Слабая	Есть намек на связь, но шум велик
0.40 – 0.60	Умеренная	Заметная связь, полезно для анализа
0.60 – 0.80	Сильная	Выраженная зависимость
0.80 – 1.00	Очень сильная	Почти функциональная связь

По количеству переменных

Тип	Описание	Применение в ИБ
Парная	Связь между двумя переменными	Базовый анализ: `requests_per_min` ↔ `cpu_load`
Частная	Связь между X и Y при фиксированном Z	Исключение влияния третьего фактора
Множественная	Связь одной переменной с несколькими	Прогнозирование `is_malicious` по набору признаков

Корреляционный анализ в реальной практике кибербезопасности решает несколько ключевых задач.

1. Отбор признаков (Feature Selection) для ML-моделей.

При построении моделей обнаружения вторжений (IDS) или классификации вредоносного ПО собираются сотни признаков. Если два признака имеют сильную корреляцию $|r| > 0.85$, они несут практически одинаковую информацию, и один из них можно удалить без потери качества модели.

Пример: "`requests_per_min`" и "`bytes_per_min`" часто имеют корреляцию 0.9. Оставив только один из этих признаков, мы упрощаем модель и снижаем риск переобучения.

2. Построение графов зависимостей (Dependency Graphs).

Корреляционная матрица всех признаков позволяет построить граф связей, где узлы — признаки, а ребра — значимые корреляции. Это помогает аналитику SOC:

- выявить скрытые паттерны атак (например, кластер признаков, которые одновременно растут при DDoS);
- понять архитектуру системы: какие метрики влияют друг на друга.

3. Детекция аномалий через нарушение корреляций.

Это одна из самых мощных техник в UEBA (User and Entity Behavior Analytics).

Строится "профиль нормальных корреляций" на исторических данных. Затем в реальном времени вычисляются скользящие корреляции и сравниваются с базовыми. Резкое отклонение генерирует сигнал аномалии.

Пример. В штатном режиме объем исходящего трафика ("`bytes_out`") и количество запросов к БД ("`db_queries`") имеют умеренную положительную корреляцию ($r \approx 0.5$).

Атака (эксфильтрация данных): злоумышленник начинает выгружать базу данных. Объем трафика резко растет, а количество запросов остается прежним (или падает, если используется прямой дамп).

Реакция системы: корреляция резко падает до $r \approx 0$ или становится отрицательной. Система фиксирует разрыв привычной связи и генерирует сигнал аномалии.

4. Расследование инцидентов (Incident Investigation).

При расследовании инцидента аналитик ищет коррелирующие события во времени:

- "В момент утечки данных какие еще события были аномальными?"
- "С каким IP-адресом коррелирует всплеск неудачных поаыток входа в систему?"

Корреляционный анализ временных рядов (cross-correlation) позволяет находить запаздывающие связи: например, сканирование портов происходит за 2 часа до основной атаки.

Для изучения технологий корреляционного анализа будет использоваться набор данных, описывающих сетевой трафик и поведение пользователей.

In [2]:

```
# @title
np.random.seed(42)
n_samples = 5000

# ============================================================
# 1. БАЗОВЫЕ НЕЗАВИСИМЫЕ ПРИЗНАКИ
# ============================================================
# Дискретное (Пуассон) — частота запросов
req_per_min = np.random.poisson(lam=15, size=n_samples)

# Равномерное — эфемерный порт источника
source_port = np.random.randint(low=49152, high=65536, size=n_samples)

# Логнормальное — размер пейлоада (тяжёлые хвосты)
payload_size = np.random.lognormal(mean=3.0, sigma=1.2, size=n_samples)

# ============================================================
# 2. ЗАВИСИМЫЕ КОЛИЧЕСТВЕННЫЕ ПРИЗНАКИ
# ============================================================
# А) ЛИНЕЙНАЯ зависимость: CPU растёт пропорционально запросам
cpu_load = np.clip(20 + 0.5 * req_per_min + np.random.normal(0, 3, n_samples), 0, 100

# Б) НЕЛИНЕЙНАЯ МОНОТОННАЯ зависимость: время отклика от размера пейлоада
# 2) Время отклика — нелинейная (степенная) зависимость + шум
#response = 10 + 0.05 * payload_size**1.3 + np.random.exponential(40, n_samples)

# Нелинейная монотонная зависимость с ЛЁГКИМИ хвостами + умеренный шум
response = 10 + 60 * np.log1p(payload_size) + np.random.exponential(20, n_samples)


# В) НЕЛИНЕЙНАЯ НЕМОНОТОННАЯ (U-образная) зависимость: джиттер от нагрузки
network_jitter = np.clip(5 + 0.03 * (req_per_min - 15)**2 +
                         np.random.exponential(0.5, n_samples), 1, 100)

# ============================================================
# 3. ЦЕЛЕВАЯ ПЕРЕМЕННАЯ
# ============================================================
attack_prob = 1 / (1 + np.exp(-(req_per_min/30 + payload_size/500 - 2)))
is_malicious = np.random.binomial(1, attack_prob)

# ============================================================
# 4. ИНЖЕКЦИЯ АНОМАЛИЙ
# ============================================================
# Выбросы: 30 «атак» — аномально большие payload И response
out_idx = np.argsort(payload_size)[-30:]
payload_size[out_idx]  *= 5    # уходят далеко в правый хвост (1700–7000 байт)
#response[out_idx] *= 10   # и вверх (1000–4000 мс)
response[out_idx] = 1.5 * payload_size[out_idx] + np.random.exponential(300, out_idx.

# Сканирование портов: 30 последовательных эфемерных портов
port_scan_idx = np.random.choice(n_samples, 30, replace=False)
is_malicious[port_scan_idx] = 1
source_port[port_scan_idx] = np.arange(49152, 49152 + 30)

# ============================================================
# 5. КАЧЕСТВЕННЫЕ ПРИЗНАКИ
# ============================================================
roles = np.random.choice(['admin', 'user', 'guest'], n_samples, p=[0.05, 0.75, 0.20])
methods = np.random.choice(['GET', 'POST', 'PUT', 'DELETE'], n_samples, p=[0.6, 0.2,

severity_categories = ['Low', 'Medium', 'High', 'Critical']
severity_level = np.empty(n_samples, dtype=object)
```

```
legit = (is_malicious == 0)
severity_level[legit]  = np.random.choice(severity_categories, legit.sum(),
                                          p=[0.60, 0.30, 0.08, 0.02])
severity_level[~legit] = np.random.choice(severity_categories, (~legit).sum(),
                                          p=[0.05, 0.20, 0.45, 0.30])

# ============================================================
# 6. СБОРКА ДАТАСЕТА
# ============================================================
df = pd.DataFrame({
    'user_role': roles,
    'request_method': methods,
    'requests_per_min': req_per_min,
    'payload_size_bytes': payload_size,
    'response_time_ms': response,
    'cpu_load_percent': cpu_load,
    'network_jitter_ms': network_jitter,
    'source_port': source_port,
    'severity_level': severity_level,
    'is_malicious': is_malicious,
})
```

In [3]:
```
df.head()
```

Out[3]:

	user_role	request_method	requests_per_min	payload_size_bytes	response_time_ms	cpu_lo
0	user	POST	18	105.831194	295.403983	
1	admin	DELETE	10	32.328640	240.614250	
2	user	DELETE	16	7.390757	193.060484	
3	user	GET	19	72.179528	298.429095	
4	user	POST	11	20.519973	200.519969	

In [10]:
```
len(out_idx)
```

Out[10]:
```
30
```

Признаки:

- `user_role` : тип пользователя;
- `request_method` : тип запроса;
- `requests_per_min` : частота запросов;
- `payload_size_bytes` : размер пейлоада;
- `response_time_ms` : время отклика;
- `cpu_load_percent` : загрузка CPU;
- `network_jitter_ms` : разница между самой большой и самой маленькой задержки при передаче пакетов за определённый период;
- `source_port` : порт источника;
- `is_malicious` : является вредоносным или нет.

2. Корреляция между количественными признаками

Рассмотрим количественные признаки нашего датасета: "`requests_per_min`", "`payload_size_bytes`", "`response_time_ms`" и "`cpu_load_percent`". Разберем, как визуализировать связи, как их измерять и как отличить линейную зависимость от нелинейной.

Визуальный анализ зависимости

Прежде чем рассчитывать численные показатели корреляции, обязательно нужно визуализировать предполагаемые зависимости. В кибербезопасности это критично, так как атаки часто формируют специфические кластеры на графиках, которые «усредняются» при расчете числовых метрик.

Основным инструментом визуального анализа зависимости количественных признаков является **диаграмма рассеяния** (точечная диаграмма, англ. **scatter plot**), которая отображает данные в виде точек на декартовой плоскости. Каждая точка представляет собой наблюдение с двумя или более признаками, где положение точки определяется их значениями.



Точечная диаграмма позволяет визуально оценить наличие, вид и силу зависимости между двумя признакамии. Анализ проводится по распределению точек на графике и их общей конфигурации.

Наличие зависимости.

Зависимость между переменными присутствует, если точки на диаграмме образуют неслучайную структуру, то есть образуют какой-либо паттерн, а не разбросаны хаотично. Если точки расположены беспорядочно и не образуют никакого явного узора, это может указывать на отсутствие взаимосвязи между переменными (нулевая корреляция).

Вид зависимости.

Вид зависимости определяется по форме распределения точек:

1. Линейная зависимость:

- положительная корреляция: точки выстраиваются примерно вдоль линии, идущей из левого нижнего в правый верхний угол диаграммы. Это означает, что при увеличении значения переменной на оси X увеличивается и значение переменной на оси Y;
- отрицательная корреляция: точки образуют линию, идущую из левого верхнего в правый нижний угол. Это указывает на то, что при увеличении значения X значение Y уменьшается.

2. Нелинейная зависимость: точки могут формировать кривые, параболы, синусоиды или другие нелинейные зависимости. Например, если точки образуют U-образную или обратную U-образную форму, это может указывать на квадратичную зависимость.

Сила зависимости.

Сила зависимости определяется по тому, насколько тесно точки группируются вокруг линии (или кривой), которая могла бы описать их распределение:

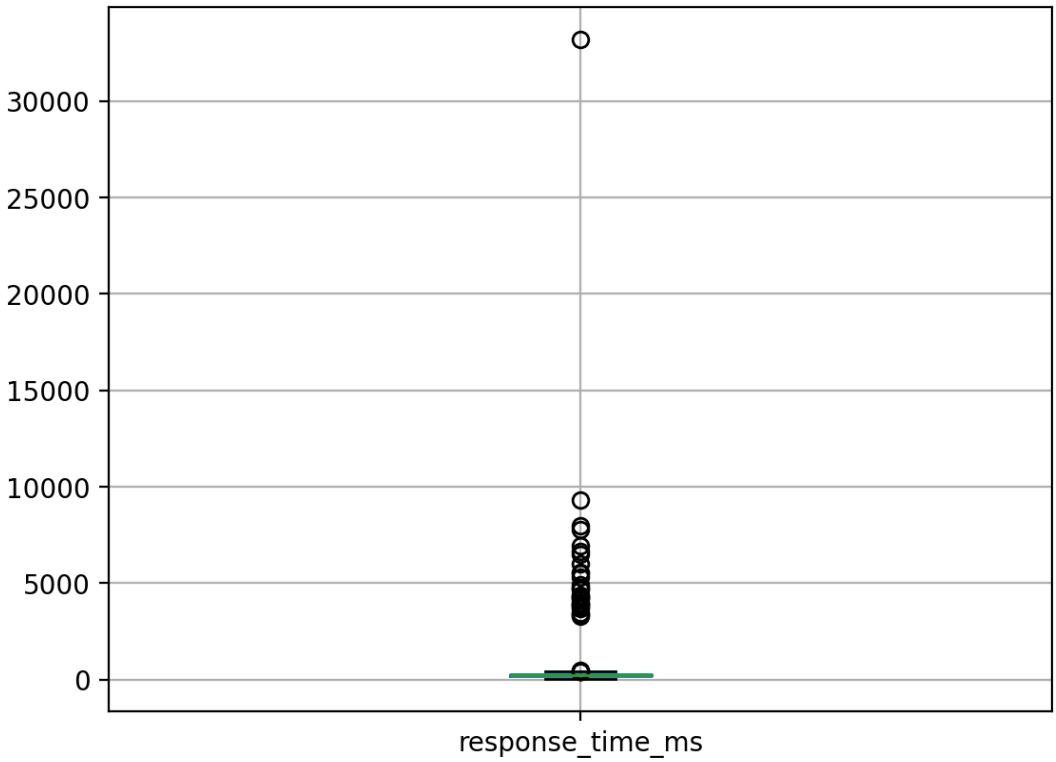
- сильная зависимость: точки плотно сгруппированы вокруг линии тренда. Чем меньше разброс точек относительно линии, тем сильнее связь между переменными;
- слабая зависимость: точки сильно разбросаны, и линия тренда лишь условно объединяет их.

Важное правило использования корреляционного анализа в кибербезопасности.

Взаимосвязи признаков исследуются на нормальном поведении (пользователя, хоста, трафика) без аномалий.

В нашем примере значительные аномалии имеет только признак "`response_time_ms`", поэтому для дальнейших расчетов нужно исключить из набора данных наблюдения, для которых значение признака превышает 1000 мс.

In [8]:
```
df[['response_time_ms']].boxplot();
```



In [5]:
```
clean = df[df['response_time_ms'] <= 1000].copy()
```

Построим диаграммы рассеяния для анализа зависимости между признаками:

- `requests_per_min` ↔ `cpu_load_percent`;
- `payload_size_bytes` ↔ `response_time_ms`;
- `requests_per_min` ↔ `network_jitter_ms`;
- `source_port` ↔ `cpu_load_percent`.

Если известно направление зависимости, то независимый признак располагается по оси абсцисс, а зависимый - по оси ординат.

Например, логично предположить, что загрузка процессора (`cpu_load_percent`) зависит от количества запросов (`requests_per_min`).

In [14]:
```
fig, axes = plt.subplots(2, 2, figsize=(10, 7))

# ============================================================
# Панель 1: requests_per_min → cpu_load_percent
# ============================================================

ax = axes[0, 0]
ax.scatter(clean['requests_per_min'], clean['cpu_load_percent'],
           s=15, alpha=0.45, color='steelblue', edgecolors='none')
ax.set_title('requests_per_min → cpu_load_percent', fontsize=12, fontweight='bold')
ax.set_xlabel('requests_per_min'); ax.set_ylabel('cpu_load_percent')
ax.grid(alpha=0.3)

# ============================================================
# Панель 2: payload_size_bytes → response_time_ms
# ============================================================

# out_idx - 30 аномально больших значений payload_size_bytes
mask = ~np.isin(np.arange(n_samples), out_idx)
ax = axes[0, 1]
```

```
# диаграмма нормальных значений признаков
ax.scatter(df['payload_size_bytes'][mask],
           df['response_time_ms'][mask], s=15, alpha=0.5,
           color='steelblue', label='чистые данные')

# диаграмма аномальных значений признаков
ax.scatter(df['payload_size_bytes'][out_idx],
           df['response_time_ms'][out_idx], s=15, alpha=0.5,
           color='red', label='выбросы (атаки)')

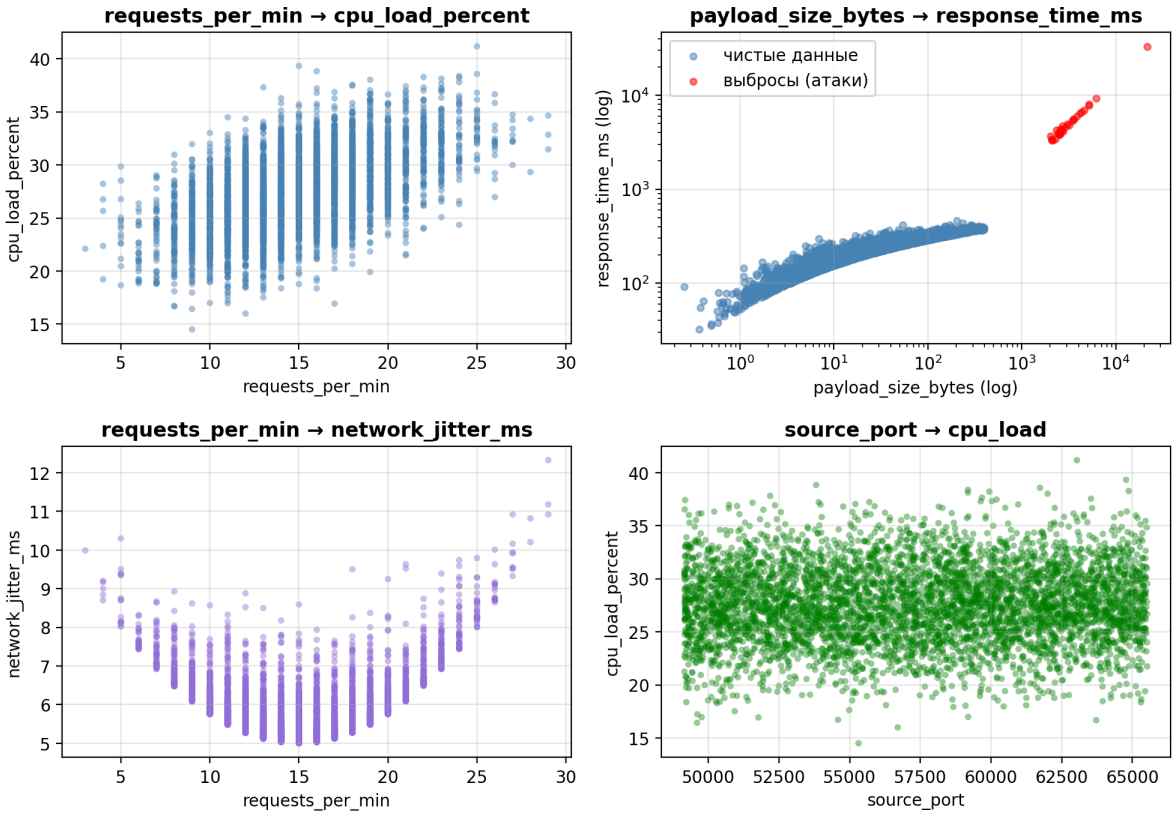
# переключает оси X и Y на логарифмическую шкалу
ax.set_xscale('log')
ax.set_yscale('log')

ax.set_xlabel('payload_size_bytes (log)')
ax.set_ylabel('response_time_ms (log)')
ax.set_title('payload_size_bytes → response_time_ms', fontweight='bold')
ax.legend()
ax.grid(alpha=0.3)

# ============================================================
# Панель 3: requests_per_min → network_jitter_ms  (U-образная)
# ============================================================
ax = axes[1, 0]
ax.scatter(clean['requests_per_min'], clean['network_jitter_ms'],
           s=15, alpha=0.45, color='mediumpurple', edgecolors='none')
ax.set_title('requests_per_min → network_jitter_ms', fontsize=12, fontweight='bold')
ax.set_xlabel('requests_per_min'); ax.set_ylabel('network_jitter_ms')
ax.grid(alpha=0.3)

# ============================================================
# Панель 4: source_port → cpu_load_percent
# ============================================================
ax = axes[1, 1]
ax.scatter(clean['source_port'], clean['cpu_load_percent'],
           s=15, alpha=0.4, color='g', edgecolors='none')
ax.set_title('source_port → cpu_load',
             fontsize=12, fontweight='bold')
ax.set_xlabel('source_port'); ax.set_ylabel('cpu_load_percent')
ax.grid(alpha=0.3)

plt.tight_layout()
```



Интерпретация диаграмм рассеяния.

Панель 1.

- Вид: облако точек плотно вытянуто вдоль прямой линии с положительным наклоном.
- Признаки линейной связи: симметричный разброс точек вокруг прямой; отсутствие изгибов, «веера» или кластеров вдали от тренда; линия проходит через центр облака на всём диапазоне.
- Вывод: зависимость линейная, положительная, умеренной силы.

Панель 2.

- Вид: плотное облако в диапазоне `payload ≈ 0.5–500 байт` (чистые данные без выбросов), `response ≈ 10–300 мс`. Чётко читается нижняя граница облака — кривая, которая почти горизонтальна при `payload < 10` (доминируют константа 10 мс и шум) и ускоряется вверх при `payload > 100`. Над кривой — «веер» точек экспоненциального шума: при малых `payload` шум сравним с детерминированной частью (большой относительный разброс), при больших — облако прижимается к кривой.
- Выбросы-атаки (красные точки), правый верхний блок: компактный диагональный кластер (`payload ≈ 2000–6000` , `response ≈ 2000–6000 мс`) плюс одна экстремальная точка (~$2 \cdot 10^4$, ~$3 \cdot 10^4$). Между облаком и кластером — пустая зона: разрыв подтверждает, что выбросы не являются продолжением естественного хвоста, а представляют мультипликативный скачок (аномалию).
- Вывод: связь нелинейная монотонная (степенная): рост `payload` даёт ускоряющийся рост `response` .

Замечание. При построении диагарммы осуществлен переход к логарифму исходных значений признаков. Это обусловлено тем, что на линейных осях 95% точек сливаются в

нечитаемое скопление у нуля, а выбросы «сжимают» масштаб.

Панель 3.

- Вид: точки образуют широкое облако в виде «чаши», но из-за шума это видно слабо. Вертикальные полосы обусловлены дискретностью признака `requests_per_min`.
- Вывод: зависимость нелинейная немонотонная (U-образная).

Панель 4.

- Вид: круглое, равномерно заполненное облако без какого-либо тренда.
- Вывод: зависимость отсутствует. Эфемерные порты генерируются ОС независимо от загрузки CPU.

Показатели зависимости

Для измерения степени зависимости между количественными признаками используются специальные показатели:

- парная корреляция:
  - парный коэффициент корреляции Пирсона;
  - коэффициент корреляции Спирмена;
  - коэффициент корреляции Кендалла;
  - корреляционное отношение;
- множественная корреляция:
  - коэффициент множественной корреляции;
  - коэффициент частной корреляции.

Коэффициенты парной корреляции

Коэффициент корреляции Пирсона

Для расчета парного коэффициента корреляции Пирсона используется выражение вида:

$$r(x,y) = \sum_{i=1}^{n} \frac{(x_i - \bar{x})(y_i - \bar{y})}{(n-1) \cdot s_x \cdot s_y},$$

где

- x_i и y_i - значения признаков i-го объекта;
- $\bar{x}$ и $\bar{y}$ - средние значения признаков;
- s_x и s_y - стандартные отклонения признаков;
- n - объем выборочной совокупности.

Парный коэффициент корреляции Пирсона позволяет оценить наличие **линейной** корреляционной зависимости.

Значение коэффициент корреляции Пирсона находится в диапазоне от -1 до 1:

- -1 указывает на сильную отрицательную корреляцию;
- 0 указывает на отсутствие корреляции;

- 1 указывает на сильную положительную корреляцию.

Для корректного применения коэффициента корреляции Пирсона обязательно должны выполняться следующие предпосылки:

- обе переменные должны быть измерены в непрерывной (интервальной или пропорциональной) шкале. Недопустимо: порядковые, номинальные, бинарные данные;
- предполагаемая зависимость между переменными - линейная. Если связь криволинейная (параболическая, экспоненциальная и т. п.), коэффициент Пирсона может быть близок к 0, даже если связь сильная;
- каждая из переменных должна быть приблизительно нормально распределена;
- значения в выборке не должны зависеть друг от друга;
- отсутствие выбросов. Экстремальные значения могут сильно исказить коэффициент;
- однородность дисперсии (гомоскедастичность). Разброс значений Y относительно линии регрессии должен быть примерно одинаковым для всех уровней X. Последствия нарушения - снижение точности оценки корреляции.

Для расчета коэффициента можно использовать метод `corr()` библиотеки `pandas`:

`DataFrame.corr(method = 'pearson', min_periods = 1, numeric_only = False)`:

- `method` - вид коэффициента корреляции (Пирсона, Спирмена, Кендалла), по умолчанию - Пирсона;
- `min_periods` - минимальное количество наблюдений, необходимое для каждой пары признаков, чтобы рассчитать коэффициент корреляции;
- `numeric_only` - включать в расчет коэффициентов корреляции только данные с плавающей запятой и целочисленные.

Альтернативным вариантоя является применение функции `pearsonr` модуля `stats` библиотеки `scipy`.

Рассмотрим технологию вычисления коэффициента корреляции Пирсона.

Сформируем корреляционную матрицу для количестенных признаков из нашего набора данных, в которой будут отображены признаки с абсолютным значением коэффициента корреляции более 0,3.

In [16]:
```
# Пары для расчёта коэффициента корреляции Пирсона
pairs = [('requests_per_min',  'cpu_load_percent',  'линейная'),
         ('payload_size_bytes','response_time_ms',  'нелинейная монотонная'),
         ('requests_per_min',  'network_jitter_ms', 'нелинейная, немонотонная (U-обра
         ('source_port',       'cpu_load_percent',  'нет связи')]

print(f"{'Признаки':38s} | {'r (Пирсон)':>12s}  | {'Форма связи по диагарамме рассеян
print("-" * 95)

for x, y, shape in pairs:
    # Используем метод .corr() из pandas для расчёта коэффициента Пирсона
    r = clean[[x, y]].corr(method='pearson').iloc[0, 1]
    print(f"{x+' ↔ '+y:38s} | {r:+12.2f}  |  ← {shape}")
```

```
Признаки                                  |   r (Пирсон)  | Форма связи по диагарамме рас
сеяния
-------------------------------------------------------------------------------------
---------
requests_per_min ↔ cpu_load_percent       |        +0.55  |  ← линейная
payload_size_bytes ↔ response_time_ms     |        +0.77  |  ← нелинейная монотонная
requests_per_min ↔ network_jitter_ms      |        +0.14  |  ← нелинейная, немонотонная
(U-образная)
source_port ↔ cpu_load_percent            |        +0.03  |  ← нет связи
```

Полученные значения коэффициента корреляции Пирсона подтверждают результаты визуального анализа диаграмм рассеяния.

Оценим коэффициент корреляции Пирсона на данных с выбросами на примере пары признаков `response_time_ms` и `payload_size_bytes`.

In [17]:
```
r_full = df[['response_time_ms', 'payload_size_bytes']].corr(method='pearson').iloc[0
print(f'r (Пирсон) на данных с выбросами: {r_full:.2f}')
```

```
r (Пирсон) на данных с выбросами: 1.00
```

Таким образом, коэффициент Пирсона позволяет корректно оценить только наличие линейной зависимости и значимо увеличивается при наличии выбросов в данных.

Если необходимо оценить статистическую значимость коэффициента корреляции, то можно использовать функцию `pearsonr` библиотеки `scipy`.

In [18]:
```
from scipy.stats import pearsonr
pearsonr(clean['response_time_ms'], clean['payload_size_bytes'])
```

Out[18]:
```
PearsonRResult(statistic=np.float64(0.7745054487047783), pvalue=np.float64(0.0))
```

Коэффициент корреляции Спирмена

В случае нарушения предпосылок применения коэффициента корреляции Пирсона, а так же при наличии монотонных зависимостей можно использовать его непараметрические аналоги - коэффициенты корреляции Спирмена или Кендалла.

Коэффициент корреляции Спирмена измеряет **монотонную связь** между двумя переменными. В отличие от коэффициента Пирсона, он работает не с исходными значениями, а с их рангами (порядковыми номерами в упорядоченном ряду).

Для расчета коэффициента корреляции Спирмена используется выражение вида:

$$r_s(x, y) = 1 - \frac{6 \cdot \sum_{i=1}^{n} (R_{xi} - R_{yi})^2}{n \cdot (n^2 - 1)},$$

где R_{xi} и R_{yi} - ранги значений признаков для i-го объекта.

Коэффициент ранговой корреляции нельзя истолковать иначе, чем сказать, что он равен коэффициенту корреляции Пирсона, вычисленному по рангам.

Коэффициент ранговой корреляции принимает значения в интервале от -1 до 1:

- $r_s = 1$ можно говорить о полной положительной корреляции признаков;

- $r_s = -1$ между признаками наблюдается полная рассогласованность. Это означает полную отрицательную корреляцию рангов;
- $r_s = 0$ корреляция между признаками отсутствует.

Предпосылками применения коэффициента Сипрмена являются:

- данные не соответствуют нормальному распределению;
- связь нелинейная, но монотонная, когда зависимость идёт «в одном направлении» (постоянно растёт или падает), но не обязательно по прямой (логарифмическая, экспоненциальная);
- переменные измерены в порядковой шкале;
- есть выбросы, поскольку расчёт идёт по рангам, экстремальные значения меньше влияют на результат, чем в коэффициенте Пирсона.

Ограничения:

- кооэффициент «не видит» немонотонных зависимостей (например, U-образных, параболических). Если связь меняет направление, может быть близок к 0, даже если связь сильная;
- много одинаковых значений признаков усложняет ранжирование. При большом числе связей коэффициент может давать огрублённую оценку;
- не подходит для номинальных данных (нельзя применять к категориям без естественного порядка (тип протокола, IP-адреса и т. п.).

Рассмотрим технологию вычисления коэффициента корреляции Спирмена.

Для его расчета можно использовать метод `corr()` библиотеки `pandas`, указав аргумент `method = 'spearman'`.

In [19]:
```
# Пары для расчёта коэффициента корреляции Спирмена
pairs = [('requests_per_min',  'cpu_load_percent',  'линейная'),
    ('payload_size_bytes','response_time_ms',  'нелинейная монотонная'),
    ('requests_per_min',  'network_jitter_ms', 'нелинейная, немонотонная (U-образная)
    ('source_port',       'cpu_load_percent',  'нет связи'),
]

print(f"{'Признаки':38s} | {'ρ (Спирмен)':>12s}  | {'Форма связи по диагарамме рассея
print("-" * 95)

for x, y, shape in pairs:
    r = clean[[x, y]].corr(method='spearman').iloc[0, 1]
    print(f"{x+' ↔ '+y:38s} | {r:+12.2f}  |  ← {shape}")
print("-" * 95)

r_full = df[['response_time_ms', 'payload_size_bytes']].corr(method='spearman').iloc[

print(f'ρ (Спирмен) payload_size_bytes ↔ response_time_ms\nна данных с выбросами: {r_
```

```
Признаки                                  |  ρ (Спирмен)  | Форма связи по диагарамме рас
сеяния
-------------------------------------------------------------------------------------
---------
requests_per_min ↔ cpu_load_percent       |        +0.53  |  ← линейная
payload_size_bytes ↔ response_time_ms     |        +0.96  |  ← нелинейная монотонная
requests_per_min ↔ network_jitter_ms      |        +0.02  |  ← нелинейная, немонотонная
(U-образная)
source_port ↔ cpu_load_percent            |        +0.03  |  ← нет связи
-------------------------------------------------------------------------------------
---------
ρ (Спирмен) payload_size_bytes ↔ response_time_ms
на данных с выбросами: 0.96
```

Таким образом, действительно коэффициент корреляции Спирмена позволяет корректно оценить наличие нелинейной монотонной заивисимости, а так является устойчивым к выбросам.

Коэффициент корреляции Кендалла

Альтерантивой коэффициенту Спирмена является коэффициент корреляции Кендалла, который так же используется при нарушении требований к коэффициенту Пирсона или для ранговых признаков.

Коэффициент корреляции Кендалла - это непараметрический показатель монотонной связи между двумя переменными, основанный на анализе согласованности пар рангов.

Для расчета коэффициента корреляции Кендалла используется выражение вида:

$$\tau = \frac{2 \cdot (P - Q)}{n \cdot (n - 1)},$$

где

- P - число совпадений изменения рангов признаков;
- Q - число несовпадений изменения рангов признаков.

Значения коэффициента корреляции Кендалла интерпретируются аналогично коэффициенту корреляции Спирмена.

Коэффициент корреляции Кендалла имеет те же предпосылки применения и ограничения, что и коэффициент корреляции Спирмена.

Рассмотрим технологию вычисления коэффициента корреляции Кендалла.

Для его расчета можно использовать метод `corr()` библиотеки `pandas`, указав аргумент `method = 'kendall'`.

In [20]:
```
# Пары для расчёта коэффициента корреляции Кендалла
pairs = [
    ('requests_per_min',  'cpu_load_percent',  'линейная'),
    ('payload_size_bytes','response_time_ms',  'нелинейная монотонная'),
    ('requests_per_min',  'network_jitter_ms', 'нелинейная, немонотонная (U-образная)'),
    ('source_port',       'cpu_load_percent',  'нет связи'),
]

print(f"{'Пара признаков':38s} | {'τ (Кендалл)':>12s}  | {'Форма связи по диагарамме
```

```
print("-" * 95)

for x, y, shape in pairs:
    r = clean[[x, y]].corr(method='kendall').iloc[0, 1]
    print(f"{x+' ↔ '+y:38s} | {r:+12.2f}  |  ← {shape}")
print("-" * 95)

r_full = df[['response_time_ms', 'payload_size_bytes']].corr(method='spearman').iloc[

print(f'τ (Кендалл) payload_size_bytes ↔ response_time_ms\nна данных с выбросами: {r_
```

```
Пара признаков                         |  τ (Кендалл)  | Форма связи по диагарамме рас
сеяния
-----------------------------------------------------------------------------------------
---------
requests_per_min ↔ cpu_load_percent    |         +0.38  |  ← линейная
payload_size_bytes ↔ response_time_ms  |         +0.84  |  ← нелинейная монотонная
requests_per_min ↔ network_jitter_ms   |         +0.01  |  ← нелинейная, немонотонная
(U-образная)
source_port ↔ cpu_load_percent         |         +0.02  |  ← нет связи
-----------------------------------------------------------------------------------------
---------
τ (Кендалл) payload_size_bytes ↔ response_time_ms
на данных с выбросами: 0.96
```

Сравнение коэффициентов парной корреляции.

Признаки	r (Пирсон)	ρ (Спирмен)	τ (Кендалл)	Интерпретация
`requests ↔ cpu_load`	+0.55	+0.53	+0.38	$r \approx \rho \rightarrow$ линейная связь; τ ниже — это норма (масштаб Кендалла)
`payload ↔ response, чистые`	+0.77	+0.96	+0.84	$\rho \gg r \rightarrow$ нелинейная монотонная (степенная) зависимость
`payload ↔ response, полные`	+1	+0.96	+0.96	r сравнялся с ρ из-за выбросов: аномалии имитируют линейность
`requests ↔ jitter`	+0.14	+0.02	+0.02	Все ≈ 0 → коэффициенты не выявляют нелинейные связи (U-форма) (нужно η)
`source_port ↔ cpu_load`	+0.03	+0.03	+0.02	Связи нет (контрольная пара)

Линейная пара requests ↔ cpu_load: все три коэффициента согласованы (с учётом разного масштаба τ) — достаточно коэффициента Пирсона.

Нелинейная монотонная связь - пара payload ↔ response, чистые: Спирмен и Кендалл показывают сильную связь (0.96 / 0.84), Пирсон — значительно меньшую (0.77).

Нелинейная монотонная связь - пара payload ↔ response, полные: выбросы искажают значение коэфициента корреляции Пирсона и формируют не правильный вывод о наличии очень сильной линейной связи. Ранговые коэффициенты меняются незначительно.

U-образная связь - пара requests ↔ jitter: все три коэффициента ≈ 0. Это не «связи нет», а «нет монотонной связи» — джиттер растёт и при простое, и при перегрузке. Парные коэффициенты здесь бессильны; диагноз ставится по диаграмме рассеяния (парабола групповых средних) и корреляционному отношению.

Отсутствие связи - пара source_port ↔ cpu_load: все коэффициенты ≈ 0, что подтверждает независимость source_port от количественных признаков.

Корреляционное отношение

Коэффициенты Пирсона и Спирмена имеют ограничения. Пирсон видит только прямую линию, Спирмен — только монотонный рост или падение.

Что делать, если зависимость сильная, однако носит явно нелинейный и немонотонный характер? К сожалению, не существует простого ответа на этот вопрос, так как не имеется естественного обобщения коэффициента корреляции Пирсона на случай нелинейных зависимостей. Поэтому на практике применяется один из следующих подходов:

1. Преобразование переменных с целью получения линейной зависимости, а затем уже вычисление корреляции между преобразованными величинами. Для этого часто используется логарифмическое преобразование.
2. Применение специального показателя - "корреляционное отношение".

Корреляционное отношение η измеряет любую зависимость (линейную, монотонную, немонотонную, пороговую).

Практическое применение корреляционного отношения в кибербезопасности аналогично рассмотренным ранее коэффифиентам корреляции.

Дополнительно корреляционное отношение позволяет отслеживать пороговые эффекты.

Многие системы безопасности работают по пороговому принципу. Например, загрузка CPU (Y) может оставаться стабильной (~20%), пока количество запросов (X) не достигнет критического порога (например, 500 req/min), после чего загрузка CPU резко возрастает до 90%.

В такой ситуации коэффициенты корреляции будут иметь низкие значения, а корреляционное отношение покажет сильную связь.

Свойства:

- $0 \leq \eta_{yx} \leq 1$;
- $\eta_{yx} = 0 \Rightarrow$ связи нет;
- $\eta_{yx} = 1 \Rightarrow$ функциональная зависимость;
- η_{yx} всегда $\geq |r|$;
- величина асимметрична : $\eta_{yx} \neq \eta_{xy}$.

Из свойства ассиметричности следует, что при расчете корреляционного отношения первоначально нужно определить зависимый признак Y и независимый X.

Диагностическая триада:

- $r \approx \rho \approx \eta$ — связь линейная;
- $r < \rho \approx \eta$ — нелинейная монотонная;
- $r \approx \rho \ll \eta$ — немонотонная (U-образная, пороговая).

Асимметрия корреляционного отношения:

- η_{yx} показывает, насколько хорошо знание x предсказывает y;
- η_{xy} показывает обратное.

Если $\eta_{yx} \gg \eta_{xy'}$ это означает направленность влияния: x лучше предсказывает y, чем наоборот.

Расчет корреляционного отношения основан на следующей идее: разбить значения независимого признака X на группы и посмотреть, какую долю общей дисперсии зависимого признака Y объясняют различия между группами.

Для расчета корреляционного отношения используются следующие выражения:

- взвешенные по количеству элементов квадраты отклонений общего среднего от внутригрупповых средних:

$$ss_{\text{межгр}} = \sum_{j=1}^{m} n_j \cdot (\bar{y}_j - \bar{y})^2$$

- общая сумма квадратов:

$$ss_{\text{общ}} = \sum_{i=1}^{n} (y_i - \bar{y})^2,$$

m - количество интервалов независимого признака;

n_j - количество значений зависимого признака, принадлежащих j-му интервалу;

$\bar{y}_j = \frac{\sum\limits_{i=1}^{m} y_{ij}}{n_j}$ - групповое среднее значение зависимого признака;

$\bar{y} = \frac{\sum\limits_{i=1}^{n} y_i}{n}$ - общее среденее значение зависимого признака.

Корреляционное отношение вычисляется как отношение вида:

$$\eta^2 = \sqrt{\frac{ss_{\text{межгр}}}{ss_{\text{общ}}}}.$$

Рассмотрим пример расчета корреляционного отношения для признаков, которые имеют достаточно сильную нелинейную зависимость.

Рассмотрим технологию расчета корреляционного отношения для пар признаков:

- `requests_per_min` (X) → `cpu_load_percent` (Y);
- `payload_size_bytes` (X) → `response_time_ms` (Y);
- `requests_per_min` (X) → `network_jitter_ms` (Y).

Ни одна библиотека `Python` не имеет в своем составе встроенную функцию для расчета корреляционного отношения.

In [21]:
```python
# Функция расчета корреляционного отношения

# Y - зависимый признак,
# X_bins — дискретный или разбитый на бины непрерывный признак
def corr_ratio(Y, X_bins):
    # Удаляем строки с NaN в бинах (пустые бины)
    mask = X_bins.notna()
    Y_clean = Y[mask]
    X_clean = X_bins[mask]
    # общая сумма квадратов
    total = np.sum((Y_clean - Y_clean.mean())**2)
    if total == 0:
        return 0.0
    # межгрупповая сумма квадратов
    # группируем только по непустым бинам
    between = sum(len(g) * (g.mean() - Y_clean.mean())**2
                  for _, g in Y_clean.groupby(X_clean)
                  if len(g) > 0)
    # корреляционное отношение
    return np.sqrt(between / total)
```

Типы независимых признаков:

- `requests_per_min` : непрерывный;
- `payload_size_bytes` : непрерывный;
- `requests_per_min` : дискретный.

Дискретизация непрерывных независимых признаков для группировки.

Используем функцию `pd.cut` .

Для признака `payload_size_bytes` важна «ширина» диапазона, поэтому применим `pd.cut` с осмысленными границами.

Для признака `requests_per_min` , который имеет перекошенное распределение, целесообразно использовать разбиение на квантильные бины так, чтобы в каждом было примерно одинаковое число наблюдений (ширина может быть различной).

In [22]:
```python
# Дискретизация непрерывных независимых признаков X для группировки
clean['req_bins'] = pd.cut(clean['requests_per_min'], bins=range(0, 40, 3))
clean['payload_bins'] = pd.qcut(clean['payload_size_bytes'], q=15)
```

In [23]:
```python
print("=== Корреляционное отношение vs Пирсон и Спирмен (чистые данные) ===")
cases = [
    ('requests_per_min','cpu_load_percent', clean['requests_per_min']),
    ('payload_size_bytes','response_time_ms', clean['payload_bins']),
    ('requests_per_min','network_jitter_ms', clean['req_bins']),
]
for x, y, xb in cases:
    r,_   = stats.pearsonr(clean[x], clean[y])
    rho,_ = stats.spearmanr(clean[x], clean[y])
    eta   = corr_ratio(clean[y], xb)
    print(f"{x:20s} → {y:20s}  r={abs(r):.2f}  ρ={abs(rho):.2f}  η={eta:.2f}")

# Асимметрия на паре payload ↔ response
```

```
eta_yx = corr_ratio(clean['response_time_ms'], clean['payload_bins'])
eta_xy = corr_ratio(clean['payload_size_bytes'],
                    pd.qcut(clean['response_time_ms'], q=15))
print(f"\nАсимметрия: η(response → payload)={eta_yx:.2f}, η(payload → response)={eta_
```

```
=== Корреляционное отношение vs Пирсон и Спирмен (чистые данные) ===
requests_per_min     → cpu_load_percent      r=0.55  ρ=0.53  η=0.55
payload_size_bytes   → response_time_ms      r=0.77  ρ=0.96  η=0.95
requests_per_min     → network_jitter_ms     r=0.14  ρ=0.02  η=0.76

Асимметрия: η(response → payload)=0.95, η(payload → response)=0.87
```

Для линейной зависимости `requests_per_min → cpu_load_percent` : все три метрики совпадают — корреляционное отношение не даёт новой информации, достаточно использование коэффициента Пирсона.

Для нелинейной монотонной зависимости `payload_size_bytes → response_time_ms` : корреляционноо отношение примрено равно коэффициенту корреляции Спирмена и не дает новой информации.

Для U-образной зависимости: коэффициенты Пирсона и Спирмена близки к нулю и могли бы убедить аналитика, что «нагрузка не влияет на джиттер». Но $\eta \approx 0.76$ доказывает обратную связь: джиттер минимален при оптимальной нагрузке ~15 req/min и растёт и при простое, и при перегрузке. В реальной системе это означает: мониторинг только линейных корреляций пропустит деградацию канала на обоих краях нагрузки.

Асимметрия: $\eta(response \to payload) > \eta(payload \to response)$ — размер пейлоада предсказывает время отклика лучше, чем время отклика позволяет восстановить размер пейлоада (шум и нелинейность «сжимают» информацию). Это подсказывает направление причинного влияния в паре.

Коэффициент частной корреляции

Коэффициент частной корреляции измеряет степень линейной связи между двумя переменными X и Y при условии, что влияние одной или нескольких других переменных исключено. Иными словами, он показывает «чистую» связь: насколько сильно и в каком направлении одна переменная влияет на другую, если мы «убираем» из рассмотрения эффект третьих факторов.

В кибербезопасности это инструмент отделения прямых причинных цепочек от ложных корреляций, которыми изобилуют логи: почти все метрики растут при общей нагрузке на систему и потому «коррелируют» между собой без прямой связи.

Коэффициенты частной корреляции могут быть различных порядков. Порядок определяется количеством учитывемых признаков, влияние которых мы исключаем. Например, при исключении влияния одного признака вычисляется коэффициент первого порядка:

$$r_{xy,z} = \frac{r_{xy} - r_{xz} \cdot r_{yz}}{\sqrt{(1 - r_{xz}^2)(1 - r_{yz}^2)}},$$

где $r_{xy},\ r_{xz},\ r_{yz}$ - коэффициенты парной корреляции Пирсона.

Интерпретация сравнения парного и частного коэффициентов корреляции.

Соотношение	Интерпретация
$r_{xy,z} \approx r_{xy}$	Связь прямая: z не является причиной зависимости между x и y
$r_{xy,z} \ll r_{xy}$ (стремится к 0)	Связь ложная: оба признака управляются z (конфаундер) — после контроля z прямая связь исчезает
$r_{xy,z} > r_{xy}$	z — супрессор: маскировал прямую связь (работал как шумовой фон); после его учёта связь между x и y становится заметнее

Рассмотрим несколько примеров использования коэффициента частной корреляции.

In [24]:
```
# Функция расчета парных коэффициентов корреляции Пирсона
def r(x, y):
    return stats.pearsonr(x, y)[0]

# Функция расчета частных коэффициентов корреляции первого порядка
def partial_formula(df, x, y, z):
    r_xy, r_xz, r_yz = r(df[x], df[y]), r(df[x], df[z]), r(df[y], df[z])
    return (r_xy - r_xz*r_yz) / np.sqrt((1 - r_xz**2) * (1 - r_yz**2))
```

Пример 1.

Гипотеза: между признаками `payload_size_bytes ⇔ response_time_ms` — прямая связь, которая сохраняется при контроле признака `requests_per_min`.

Гипотеза-конкурент: `payload_size_bytes` и `response_time_ms` связаны лишь потому, что оба растут с общей интенсивностью запросов (`requests_per_min`).

In [25]:
```
r_xy  = r(clean['payload_size_bytes'],
          clean['response_time_ms'])

r_par = partial_formula(clean,
                        'payload_size_bytes',
                        'response_time_ms',
                        'requests_per_min')

print(f"Парная r(payload ⇔ response)            = {r_xy:+.3f}")
print(f"Частная r(payload ⇔ response | requests) = {r_par:+.3f}")
```

```
Парная r(payload ⇔ response)            = +0.775
Частная r(payload ⇔ response | requests) = +0.774
```

Вывод: частная корреляция ≈ парной ⇒ связь «размер пейлоада → время обработки» прямая (физика сервера), а не артефакт общей нагрузки.

Пример 2.

Гипотеза: между признаками `cpu_load_percent ⇔ is_malicious` — прямая связь, которая сохраняется при контроле признака `requests_per_min`.

Гипотеза-конкурент: оба признака растут с интенсивностью запросов: CPU — линейно, вероятность атаки — через логистическую функцию. Предполагается ложная корреляция, которая должна исчезнуть.

In [26]:
```
r_xy  = r(df['cpu_load_percent'], df['is_malicious'])
r_par = partial_formula(df, 'cpu_load_percent', 'is_malicious', 'requests_per_min')
```

```
print(f"Парная r(cpu ⇔ is_malicious) = {r_xy:+.3f}")
print(f"Частная r(cpu ⇔ is_malicious | requests) = {r_par:+.3f})")
```

```
Парная r(cpu ⇔ is_malicious) = +0.026
Частная r(cpu ⇔ is_malicious | requests) = +0.002)
```

Оба коэффициента ≈ 0: загрузка CPU не маркирует вредоносность, независимые признаки остаются независимыми и после контроля.

Пример зависимости двух признаков, обусловленной тертьим признаком.

In [27]:
```
n = 5000
# Синтетическая тройка: Z — общая причина X и Y
Z = np.random.normal(0, 1, n)
X = 0.8*Z + np.random.normal(0, 0.6, n)
Y = 0.8*Z + np.random.normal(0, 0.6, n)

r_xy, r_xz, r_yz = [stats.pearsonr(a, b)[0] for a, b in [(X,Y),(X,Z),(Y,Z)]]
partial_r_xyz = (r_xy - r_xz * r_yz) / np.sqrt((1 - r_xz**2) * (1 - r_yz**2))

print(f"парная r(X ⇔ Y)   = {r_xy:+.3f}")
print(f"частная r(X ⇔ Y | Z) = {partial_r_xyz:+.3f}")
```

```
парная r(X ⇔ Y)   = +0.635
частная r(X ⇔ Y | Z) = -0.009
```

Парная корреляция ≈ +0.64, частная ≈ 0 — связь целиком обусловлена общим драйвером фактором Z.

Коэффициент множественной корреляции

На практике часто возникает задача оценивания корреляции между зависимым признаком и несколькими независимыми признаками.

Для этого используется коэффициент множественной корреляции:

$$R(y, [x_1, x_2, \ldots, x_m]) = \mathbf{v}^{\mathrm{T}} \cdot \mathbf{R}^{-1} \cdot \mathbf{v},$$

где

- $\mathbf{v}$ - вектор коэффициентов парной корреляции зависимого признака y с независимыми $x_1, x_2, \ldots, x_m$;
- $\mathbf{R}$ - матрица парных коэффициентов корреляции между независимыми признаками.

1. Коэффициент множественной корреляции заключен в пределах от 0 до 1.
2. Включение в расчет коэффициента множественной корреляции нового независимого признака не может уменьшить его значение.
3. С помощью коэффициента множественной корреляции нельзя сделать вывод о направлении связи между переменными. Только в частном случае, когда все коэффициенты парной корреляции имеют одинаковый знак, этот знак можно отнести и к коэффициенту множественной корреляции и утверждать о соответствующем направлении множественной связи.
4. Измеряет только линейную зависимость.

Рассмотрим несколько примеров вычисления коэффициента множественной корреляции.

Пример 1 — линейная связь и избыточность признака.

In [28]:
```
# Функция расчета R через матрицу парных корреляций
def mult_r_matrix(df, y, xs):
    Rm = df[xs + [y]].corr().to_numpy()
    Rxx, ryx = Rm[:-1, :-1], Rm[:-1, -1]
    r2 = ryx @ np.linalg.inv(Rxx) @ ryx
    return r2
```

In [29]:
```
r2 = mult_r_matrix(df, 'cpu_load_percent', ['requests_per_min'])
r2_1 = mult_r_matrix(df, 'cpu_load_percent', ['requests_per_min', 'payload_size_bytes
print(f'Коэффициент парной корреляции: {r2:.3f}')
print(f'Коэффициент множественной корреляции: {r2_1:.3f}')
```

```
Коэффициент парной корреляции: 0.303
Коэффициент множественной корреляции: 0.303
```

Добавление признака `payload_size_bytes` не меняет коэффициент парной корреляции.

Вывод: нагрузка CPU объясняется интенсивностью запросов; `payload` — избыточный предиктор.

Пример 2 — добавление полиномиального расширения выявляет нелинейную связь.

Ранее было установлено, что между признаками `network_jitter_ms` и `requests_per_min` существует нелинейная U-образная связь.

Коэффициент парной корреляции Пирсона подобные зависимости не выявляет. Но если применить коэффициент множественной корреляции, добавив к признаку `requests_per_min` дополнительно квадратичный член, то это позволяет выявить зависимость.

In [30]:
```
df['req2'] = df['requests_per_min']**2
r2 = mult_r_matrix(df, 'network_jitter_ms', ['requests_per_min'])
r2_1 = mult_r_matrix(df, 'network_jitter_ms', ['requests_per_min', 'req2'])
print(f'Коэффициент парной корреляции: {r2:.3f}')
print(f'Коэффициент множественной корреляции: {r2_1:.3f}')
```

```
Коэффициент парной корреляции: 0.020
Коэффициент множественной корреляции: 0.630
```

3. Корреляция между категориальными признаками

В задачах кибербезопасности часто приходится анализировать связи между категориальными признаками. Например, существует ли зависимость между типом атаки и целевой системой, между географическим расположением и типом вредоносного ПО, между уровнем критичности инцидента и временем суток.

Технологии анализа корреляций между категориальными признаками будем рассматривать на наборе данных, содержащим информацию о киберинцидентах с

различными категориями признаков.

In [31]:
```
# @title
np.random.seed(42)

# Генерация данных с реалистичными зависимостями
n_samples = 500

# Типы атак и целевые системы с зависимостью
attack_types = np.random.choice(
    ['Phishing', 'DDoS', 'Malware', 'Brute_Force', 'SQL_Injection'],
    size=n_samples,
    p=[0.30, 0.25, 0.20, 0.15, 0.10]
)

target_systems = []
for attack in attack_types:
    if attack == 'Phishing':
        # Фишинг чаще направлен на рабочие станции
        target = np.random.choice(['Workstation', 'Server', 'Cloud', 'Mobile'], p=[0.
    elif attack == 'DDoS':
        # DDoS чаще направлен на серверы и облачные сервисы
        target = np.random.choice(['Workstation', 'Server', 'Cloud', 'Mobile'], p=[0.
    elif attack == 'Malware':
        # Вредоносное ПО чаще заражает рабочие станции и мобильные устройства
        target = np.random.choice(['Workstation', 'Server', 'Cloud', 'Mobile'], p=[0.
    elif attack == 'Brute_Force':
        # Брутфорс направлен на серверы
        target = np.random.choice(['Workstation', 'Server', 'Cloud', 'Mobile'], p=[0.
    else:  # SQL_Injection
        # SQL-инъекции направлены на серверы и облако
        target = np.random.choice(['Workstation', 'Server', 'Cloud', 'Mobile'], p=[0.
    target_systems.append(target)

# Географическое положение с зависимостью от типа атаки
geo_locations = []
for attack in attack_types:
    if attack in ['Phishing', 'Brute_Force']:
        # Фишинг и брутфорс чаще из Восточной Европы и Азии
        geo = np.random.choice(['North_America', 'Europe', 'Asia', 'Other'], p=[0.20,
    elif attack in ['DDoS', 'SQL_Injection']:
        # DDoS и SQL-инъекции чаще из Азии и Северной Америки
        geo = np.random.choice(['North_America', 'Europe', 'Asia', 'Other'], p=[0.35,
    else:  # Malware
        # Вредоносное ПО равномерно распределено
        geo = np.random.choice(['North_America', 'Europe', 'Asia', 'Other'], p=[0.25,
    geo_locations.append(geo)

# Уровень критичности с зависимостью от типа атаки
severity_levels = []
for attack in attack_types:
    if attack == 'DDoS':
        severity = np.random.choice(['Low', 'Medium', 'High', 'Critical'], p=[0.10, 0
    elif attack == 'SQL_Injection':
        severity = np.random.choice(['Low', 'Medium', 'High', 'Critical'], p=[0.05, 0
    elif attack == 'Malware':
        severity = np.random.choice(['Low', 'Medium', 'High', 'Critical'], p=[0.15, 0
    elif attack == 'Phishing':
        severity = np.random.choice(['Low', 'Medium', 'High', 'Critical'], p=[0.25, 0
    else:  # Brute_Force
        severity = np.random.choice(['Low', 'Medium', 'High', 'Critical'], p=[0.30, 0
    severity_levels.append(severity)
```

```
# Время суток (категоризированное)
time_periods = np.random.choice(
    ['Morning', 'Afternoon', 'Evening', 'Night'],
    size=n_samples,
    p=[0.20, 0.35, 0.30, 0.15]
)

# Создание DataFrame
df = pd.DataFrame({
    'attack_type': attack_types,
    'target_system': target_systems,
    'geo_location': geo_locations,
    'severity': severity_levels,
    'time_period': time_periods
})
```

In [32]:
```
df.head()
```

Out[32]:

	attack_type	target_system	geo_location	severity	time_period
0	DDoS	Cloud	North_America	High	Afternoon
1	SQL_Injection	Server	Europe	High	Afternoon
2	Malware	Workstation	Other	Low	Night
3	Malware	Mobile	Asia	Medium	Afternoon
4	Phishing	Server	Asia	Medium	Afternoon

Признаки:

- `attack_type` : тип атаки;
- `target_system` : целевая система;
- `geo_location` : географическое положение;
- `severity` : уровень критичности инцидента;
- `time_period` : время суток.

Визуальный анализ зависимостей

Визуальный анализ — первый этап разведочного анализа данных (EDA) в SOC и при расследовании инцидентов.

Типовые задачи:

- Анализ матрицы атак MITRE ATT&CK: тактика (Initial Access, Execution, Lateral Movement) × техника (Spearphishing, PowerShell, Pass-the-Hash). Аналитик визуально оценивает, какие техники чаще встречаются в рамках каких тактик.
- Профилирование инцидентов: тип атаки × целевой сегмент сети. Позволяет выявить, какие сегменты наиболее уязвимы для конкретных типов угроз.
- Оценка критичности: тип атаки × уровень критичности. Помогает расставить приоритеты при реагировании.
- Географический анализ: страна-источник × тип вредоносного ПО. Используется в Threat Intelligence для атрибуции атак.

Основными средствами визуального анализа зависимости двух категориальных признаков являются столбчатые диаграммы.

Рассмотрим пример визуального оценивания зависимости признаков `attack_type` (тип атаки) и `severity` (уровень критичности).

Для удобства построения столбчатой диаграммы рекомендуется использовать таблицу сопряженности или сводную таблицу.

In [39]:
```
# Таблица сопряженности
contingency_table = pd.crosstab(df['attack_type'], df['severity'], normalize='index')
contingency_table
```

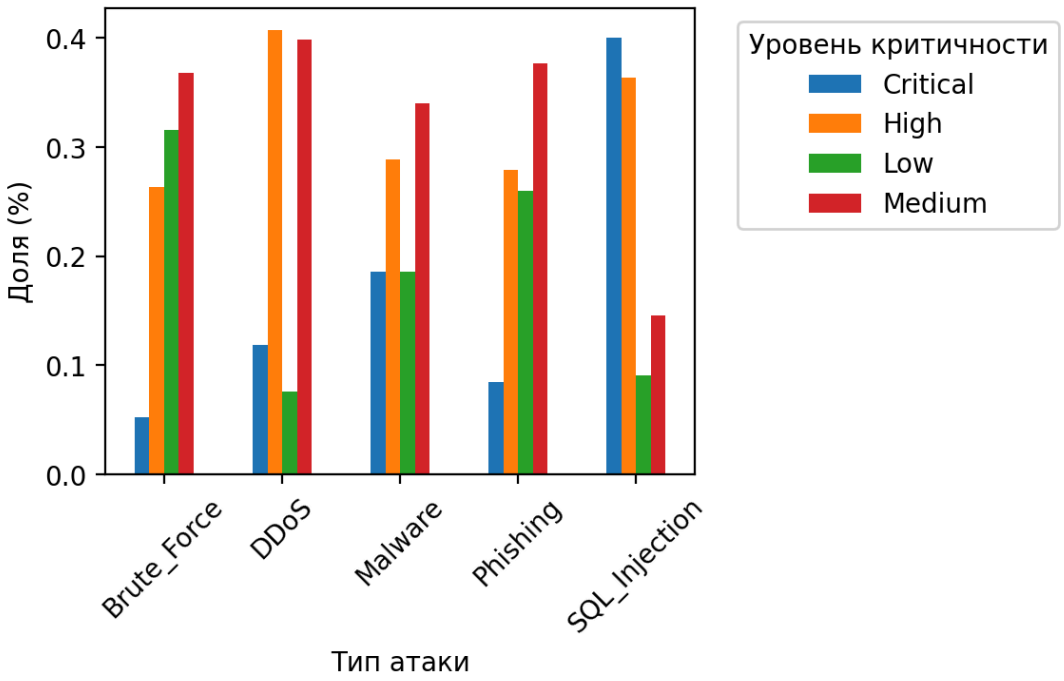
Out[39]:

severity	Critical	High	Low	Medium
attack_type				
Brute_Force	0.052632	0.263158	0.315789	0.368421
DDoS	0.118644	0.406780	0.076271	0.398305
Malware	0.185567	0.288660	0.185567	0.340206
Phishing	0.084416	0.279221	0.259740	0.376623
SQL_Injection	0.400000	0.363636	0.090909	0.145455

In [40]:
```
fig, axes = plt.subplots(figsize=(6, 4))

contingency_table.plot(kind='bar', ax=axes)
axes.set_title('Относительные частоты: Тип атаки × Уровень критичности (%)',
               fontsize=10)
axes.set_xlabel('Тип атаки')
axes.set_ylabel('Доля (%)')
axes.legend(title='Уровень критичности',
            bbox_to_anchor = (1.05, 1))
axes.tick_params(axis='x', rotation=45)
plt.tight_layout()
```

Относительные частоты: Тип атаки × Уровень критичности (%)



Как интерпретировать результаты визуализации?

Если бы признаки были независимы, высота столбцов одного цвета была бы примерно одинаковой во всех группах (или следовала бы общей средней доле по всему датасету).

Для нашего примера на диаграмме отчётливо видны устойчивые паттерны, которые не могли бы возникнуть при случайном распределении:

- `SQL_Injection` имеет уникальный профиль: резкий пик по категории `Critical` (≈ 40 %) при очень низкой доле `Low`. Это говорит о том, что тип атаки напрямую предопределяет высокую оценку риска.
- `DDoS` демонстрирует другой профиль: доминируют `High` (≈ 40 %) и `Medium` (≈ 40 %), а `Critical` и `Low` заметно ниже. Для `DDoS` критичность чаще связывают с влиянием на доступность, а не с компрометацией данных.
- `Brute_Force` — третий профиль: преобладание `Medium` и `Low`, минимальная доля `Critical` (≈ 6 %). Это отражает восприятие брутфорса как рутинной угрозы.
- `Malware` и `Phishing` имеют свои сбалансированные, но отличные друг от друга распределения.

Для нашего примера видны систематические перекосы, специфичные для каждого типа атаки, что позволяет сделать вывод о зависимости этих признаков.

Что это значит для приоритизации в SOC?

- `SQL_Injection` — главный кандидат на немедленное реагирование: у него самая высокая доля `Critical` и заметный вклад `High`.
- `DDoS` — требует отдельного соглашение об уровне обслуживания (Service Level Agreement — SLA) по доступности: основная масса инцидентов — `High` / `Medium`, но их влияние на бизнес может быть значительным.
- `Brute_Force` — можно автоматизировать обработку: большая часть инцидентов — `Low` / `Medium`, `Critical` почти нет.

- `Malware` и `Phishing` — требуют сбалансированного подхода: в них есть и `Critical`, и заметные доли `High` / `Medium`, поэтому нужна комбинация автоматических правил и ручной проверки.

Таким образом, диаграмма не просто показывает распределение, а обосновывает необходимость дифференцированных правил обработки инцидентов в зависимости от типа атаки.

На практике в качестве альтернативы часто используется мозаичный график, в котором площади прямоугольников пропорциональны частотам.

При независимости все прямоугольники в пределах одной строки имеют одинаковую ширину. Отклонения указывают на зависимость.

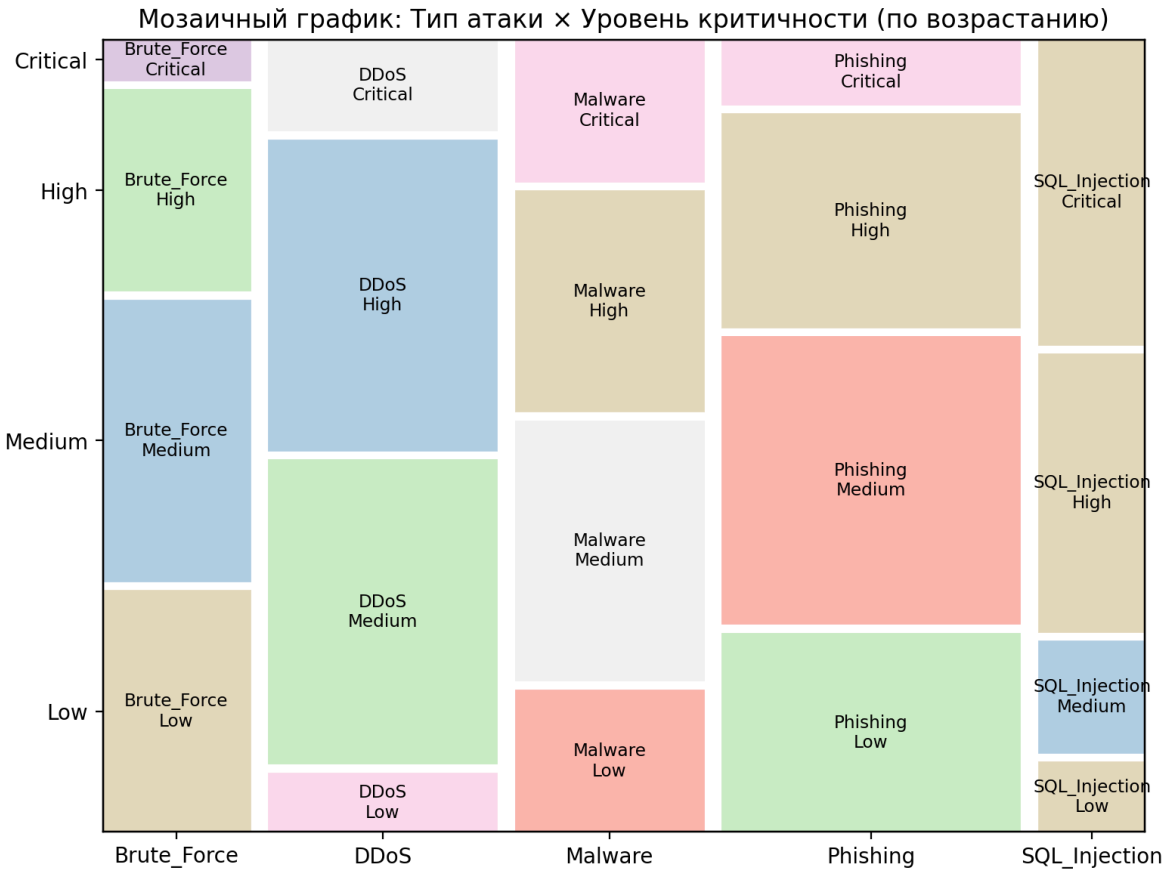
In [42]:
```
from statsmodels.graphics.mosaicplot import mosaic

# Упорядочивание уровней критичности по возрастанию
severity_order_asc = ['Low', 'Medium', 'High', 'Critical']
df['severity'] = pd.Categorical(df['severity'], categories=severity_order_asc, ordere

# Настройка цветов и добавление зазоров (gap) для четких границ
props = lambda key: {'color': plt.cm.Pastel1(np.random.rand())}

fig, ax = plt.subplots(figsize=(8, 6))
# Сортируем DataFrame, чтобы mosaic plot следовал заданному категориальному порядку
mosaic(df.sort_values(['attack_type', 'severity']), ['attack_type', 'severity'], ax=a
       title='Мозаичный график: Тип атаки × Уровень критичности (по возрастанию)',
       properties=props,
       gap=0.02)

plt.tight_layout()
```



Проверка гипотезы о независимости признаков

Наиболее распространенным критерием для проверки гипотезы о независимости двух категориальных признаков является критерий хи-квадрат (χ^2) Пирсона.

Для его использования предварительно необходимо сформировать таблицу сопряженности, в которой записываются наблюдаемые частоты.

Критерий основан на сравнении наблюдаемых и ожидаемых (при условии независимости признаков) частот.

Основные гипотезы:

H_0 (нулевая): признаки независимы;

H_1 (альтернативная): признаки зависимы.

Статистика хи-квадрат:

$$\chi^2_{\text{набл.}} = \sum_{i=1}^{m}\sum_{j=1}^{k}\frac{\left(O_{i,j} - E_{i,j}\right)^2}{E_{i,j}},$$

где:

- m, k - количество категорий первого и второго признаков соответственно;
- $O_{i,j}$ - наблюдаемая частота в ячейке (i, j);
- $E_{i,j}$ - ожидаемая частота в ячейке (i, j):

  $E_{i,j} = \frac{(\text{сумма частот по строке } i)\cdot(\text{сумма частот постолбцу } j)}{\text{общее число наблюдений}}.$

Степени свободы: $df = (m-1)\cdot(k-1)$.

Задается уровень значимости α (по умолчанию 0.05).

Для вычисленного значения $\chi^2_{\text{набл.}}$ находится p-уровень.

Если $p < \alpha$ - гипотеза H_0 отклоняется (между признаками есть зависимость).

Если $p \geq \alpha$ - гипотеза H_0 принимается (нет зависимости между признаками).

Проверим с помощью критерия χ^2 гипотезу о независимости признаков `attack_type` и `severity` .

Критерий χ^2 в `Python` реализован в классе `chi2_contingency()` модуля `stats` библиотеки `scipy` .

In [43]:
```python
# Функция расчета критерия хи-квадрат
from scipy.stats import chi2_contingency
def chi_square_independence_test(df, var1, var2, alpha = 0.05):
    # Таблица сопряжённости
    ct = pd.crosstab(df[var1], df[var2])
    n_total = ct.values.sum()
```

```
    print(f"\nГипотезы:")
    print(f"  H₀: {var1} и {var2} независимы")
    print(f"  H₁: {var1} и {var2} зависимы")

    # Расчёт
    chi2, p_value, dof, expected = chi2_contingency(ct)
    chi2_crit = stats.chi2.ppf(1 - alpha, dof)

    print(f"\nТаблица сопряжённости:")
    print(ct.to_string())

    print(f"\nРезультаты:")
    print(f"  χ² = {chi2:.4f}")
    print(f"  p-value = {p_value:.4e}")

    # Проверка условий
    min_exp = expected.min()
    pct_below_5 = (expected < 5).sum() / expected.size * 100
    print(f"\nУсловия применимости:")
    print(f"  Мин. ожидаемая частота: {min_exp:.2f}")
    print(f"  Доля ячеек с E < 5: {pct_below_5:.1f}%")
    if pct_below_5 > 20:
        print("Условия нарушены — рекомендуется точный критерий Фишера")
    else:
        print("Условия выполнены")

    # Решение
    print(f"\nРешение (α = {alpha}):")
    if p_value < alpha:
        print(f"  p-value ({p_value:.4e}) < α → ОТВЕРГАЕМ H₀")
        print(f"  Признаки статистически ЗАВИСИМЫ")
    else:
        print(f"  p-value ({p_value:.4e}) ≥ α → НЕ ОТВЕРГАЕМ H₀")
        print(f"  Нет оснований считать признаки зависимыми")

    return chi2, p_value, expected
```

In [44]:

```
# Применение функции для проверки гипотезы о независимости.
chi2, p, exp = chi_square_independence_test(
    df, 'attack_type', 'severity')
```

```
Гипотезы:
  H₀: attack_type и severity независимы
  H₁: attack_type и severity зависимы

Таблица сопряжённости:
severity       Low  Medium  High  Critical
attack_type
Brute_Force     24      28    20         4
DDoS             9      47    48        14
Malware         18      33    28        18
Phishing        40      58    43        13
SQL_Injection    5       8    20        22

Результаты:
  χ² = 69.1353
  p-value = 4.6471e-10

Условия применимости:
  Мин. ожидаемая частота: 7.81
  Доля ячеек с E < 5: 0.0%
Условия выполнены

Решение (α = 0.05):
  p-value (4.6471e-10) < α → ОТВЕРГАЕМ H₀
  Признаки статистически ЗАВИСИМЫ
```

Показатели зависимости

Выбор показателя зависимости между категориальными признаками зависит от количества категорий.

Признаки с двумя категориями

Для исследования степени тесноты связи между двумя качественными показателями, каждый из которых представлен двумя категорями используются следующие показатели:

- коэффициент ассоциации Юла;
- Коэффициент взаимной сопряженности Пирсона (φ-коэффициент).

Для расчета этих показателей предварительно необходимо сформировать таблицу сопряженности.

Признак 1	**Признак 2**		
	категория 1	*категория 2*	*маргинальные частоты*
категория 1	n_{11}	n_{12}	n_{1j}
категория 2	n_{21}	n_{22}	n_{2j}
маргинальные частоты	n_{i1}	n_{i2}	n

Коэффициент ассоциации Юла (Yule's Q)

Коэффициент ассоциации Юла вычисляется по выражению вида:

$$Q = \frac{n_{11} \cdot n_{22} - n_{12} \cdot n_{21}}{n_{11} \cdot n_{22} + n_{12} \cdot n_{21}}.$$

Свойства:

- Диапазон: $[-1, +1]$;
- $Q = 0$: отсутствие ассоциации;
- $Q > 0$: положительная ассоциация (категория 1 чаще сочетается с категорией 2);
- $Q < 0$: отрицательная ассоциация;
- $|Q| = 1$: полная ассоциация.

Для демонстрации использования коэффициента Юла создадим бинарные признаки:

- `is_ddos` : DDoS атака (да/нет);
- `is_server` : сервер (да/нет).

Оценим, есть ли между ними заивисимость.

In [45]:
```
df_bin = df.copy()
df_bin['is_ddos'] = (df_bin['attack_type'] == 'DDoS').astype(int)
df_bin['is_server'] = (df_bin['target_system'] == 'Server').astype(int)

# Таблица сопряженности 2×2
table_2x2 = pd.crosstab(df_bin['is_ddos'], df_bin['is_server'],
                        rownames=['is_ddos'], colnames=['is_server'])
table_2x2
```

Out[45]:

is_server	0	1
is_ddos		
0	236	146
1	58	60

In [46]:
```
# Функция для расчета коэффициента Юла
def yule_q(table_2x2):
    # проверка поступления на вход DataFrame
    if isinstance(table_2x2, pd.DataFrame):
        # преобразование в массив numpy
        t = table_2x2.values
    else:
        t = np.array(table_2x2)
    if t.shape != (2, 2):
        raise ValueError("Коэффициент Юла определён только для таблиц 2×2")
    a, b, c, d = t[0, 0], t[0, 1], t[1, 0], t[1, 1]
    denom = a * d + b * c
    if denom == 0:
        return 0.0
    return (a * d - b * c) / denom
```

In [47]:
```
q = yule_q(table_2x2)

print(f"\nКоэффициент Юла Q = {q:.4f}")
```

```
print(f"\nИнтерпретация:")
if abs(q) < 0.1:
    strength = "очень слабая / отсутствует"
elif abs(q) < 0.3:
    strength = "слабая"
elif abs(q) < 0.5:
    strength = "умеренная"
elif abs(q) < 0.7:
    strength = "сильная"
else:
    strength = "очень сильная"
direction = "положительная" if q > 0 else "отрицательная"
print(f"  Сила связи: {strength}")
print(f"  Направление: {direction}")
```

```
Коэффициент Юла Q = 0.2515

Интерпретация:
  Сила связи: слабая
  Направление: положительная
```

Коэффициент взаимной сопряженности Пирсона (φ-коэффициент)

φ-коэффициент — мера связи между двумя категориальными признаками, основанная на статистике хи-квадрат:

$$\varphi = \sqrt{\frac{\chi^2}{n}}.$$

Его можно использовать для признаков с любым числом категорий.

Для таблиц 2 × 2 он имеет эквивалентную формулу через элементы таблицы:

$$\varphi = \frac{n_{11} \cdot n_{22} - n_{12} \cdot n_{21}}{\sqrt{n_{1j} \cdot n_{2j} \cdot n_{i1} \cdot n_{i2}}}.$$

Свойства:

- Диапазон: $[-1, +1]$;
- $\varphi = 0$: отсутствие зависимости;
- $\varphi > 0$: положительная зависимость;
- $\varphi < 0$: отрицательная зависимость;
- $|\varphi| = 1$: полная зависимость.

Для двух бинарных переменных φ-коэффициент тождественно равен коэффициенту корреляции Пирсона.

Оценим заивисимость признаков `is_ddos` и `is_server`.

In [48]:
```
table_2x2 = pd.crosstab(df_bin['is_ddos'], df_bin['is_server'])
a, b = table_2x2.values[0]
c, d = table_2x2.values[1]
n = table_2x2.values.sum()

phi  = (a*d - b*c) / np.sqrt((a+b)*(c+d)*(a+c)*(b+d))
chi2, p, _, _ = chi2_contingency(table_2x2)
phi_chi2     = np.sqrt(chi2 / n)
phi_corr     = df_bin['is_ddos'].corr(df_bin['is_server'])
```

```
print("=" * 60)
print("ϕ-КОЭФФИЦИЕНТ ДЛЯ ТАБЛИЦЫ 2×2: DDoS × Server")
print("=" * 60)
print(f"\nСпособ 1 (формула через a,b,c,d): ϕ = {phi:.4f}")
print(f"Способ 2 (корреляция Пирсона):      r = {phi_corr:.4f}")

print(f"\nИнтерпретация:")
print(f"  |ϕ| = {abs(phi):.4f}")
if abs(phi) < 0.1:
    print(f"  → Пренебрежимо слабая связь")
elif abs(phi) < 0.3:
    print(f"  → Слабая связь")
elif abs(phi) < 0.5:
    print(f"  → Умеренная связь")
else:
    print(f"  → Сильная связь")
```

```
============================================================
ϕ-КОЭФФИЦИЕНТ ДЛЯ ТАБЛИЦЫ 2×2: DDoS × Server
============================================================

Способ 1 (формула через a,b,c,d): ϕ = 0.1089
Способ 2 (корреляция Пирсона):      r = 0.1089

Интерпретация:
  |ϕ| = 0.1089
  → Слабая связь
```

В библиотеке `scipy` для расчета коэффициента сопряжённости Пирсона используется класс `association` с параметром `method = 'pearson'`.

In [49]:
```
from scipy.stats.contingency import association
association(table_2x2, method = 'pearson')
```

Out[49]:
```
0.10829865053028875
```

Признаки с любым числом категорий

Для оценивания тесноты связи между категориальными признаками, принимающими любое количество значений используются коэффициенты взаимной сопряженности:

- коэффициент контингенции Пирсона;
- коэффициент сопряженности Крамера;
- коэффициента взаимной сопряженности Чупрова.

Для их расчета исходные данные должны быть представлены в виде таблиц сопряженности (m - количество строк, k - количество столбцов).

Коэффициент контингенции Пирсона (Pearson's Contingency Coefficient)

Коэффициент контингенции Пирсона вычисляется по выражению:

$$C = \sqrt{\frac{\chi^2}{\chi^2 + n}}.$$

Свойства:

- Диапазон: $[0, C_{max})$, где $C_{max} = \sqrt{\frac{q-1}{q}}$, $q = min(m, k)$;
- Для таблицы 2 × 2: $C_{max} = \sqrt{0.5} \approx 0.707$;
- Для таблицы 5 × 4: $C_{max} = \sqrt{3/4} \approx 0.866$;
- Важно: C никогда не достигает 1, что затрудняет интерпретацию.

Абсолютное значение коэффициента C не имеет универсальной шкалы, потому что максимум зависит от размерности таблицы. Поэтому интерпретация силы связи должна быть относительной:

- $C/C_{max} < 0.2$ — слабая;
- $0.2 < C/C_{max} < 0.7$ — умеренная;
- $C/C_{max} > 0.7$ — сильная.

Оценим с помощью коэффициента контингенции Пирсона зависимость признаков `attack_type` и `severity`.

In [50]:
```
# Функция для расчёта коэффициента контингенции Пирсона
def pearson_contingency_coefficient(ct):
    chi2, p, dof, exp = chi2_contingency(ct)
    n = ct.values.sum()
    C = np.sqrt(chi2 / (chi2 + n))
    k = min(ct.shape)
    C_max = np.sqrt((k - 1) / k)
    return C, C_max, chi2, p

# Вычисление коэффициента для attack_type × severity
print("=" * 65)
print("КОЭФФИЦИЕНТ КОНТИНГЕНЦИИ ПИРСОНА: attack_type × severity")
print("=" * 65)

ct = pd.crosstab(df['attack_type'], df['severity'])
C, C_max, chi2, p = pearson_contingency_coefficient(ct)

print(f"\nχ² = {chi2:.4f}, n = {ct.values.sum()}, p = {p:.4e}")
print(f"C = {C:.4f}")
print(f"C_max (для таблицы {ct.shape[0]}×{ct.shape[1]}) = {C_max:.4f}")
print(f"Относительная величина: C / C_max = {C / C_max:.4f}")

print(f"\nИнтерпретация:")
print(f"  C = {C:.4f} при максимуме {C_max:.4f}")
if C / C_max < 0.2:
    print(f"  → Связь слабая")
elif C / C_max < 0.5:
    print(f"  → Связь умеренная")
else:
    print(f"  → Связь сильная")
```

```
=================================================================
КОЭФФИЦИЕНТ КОНТИНГЕНЦИИ ПИРСОНА: attack_type × severity
=================================================================

χ² = 69.1353, n = 500, p = 4.6471e-10
C = 0.3485
C_max (для таблицы 5×4) = 0.8660
Относительная величина: C / C_max = 0.4024

Интерпретация:
  C = 0.3485 при максимуме 0.8660
  → Связь умеренная
```

Коэффициент сопряжённости Крамера (Cramér's V)

Коэффициент взаимной сопряженности Крамера вычисляется по выражению:

$$V = \sqrt{\frac{\chi^2}{n \cdot \min(m-1,\ k-1)}}.$$

Свойства:

- Диапазон: [0, 1];
- $V = 0$: полная независимость;
- $V = 1$: полная ассоциация (каждая строка соответствует только одному столбцу).

Рекомендуемые пороги для интерпретации:

- менее 0.1: пренебрежимо слабая связь;
- 0.1 – 0.3: слабая связь;
- 0.3 – 0.5: умеренная связь;
- более 0.5: сильная связь.

Оценим с помощью коэффициента взаимной сопряженности Крамера зависимость признаков `attack_type` и `severity`.

In [51]:
```
# Функция для расчёта коэффициента сопряжённости Крамера
def cramers_v(ct):
    chi2, p, dof, exp = chi2_contingency(ct)
    n = ct.values.sum()
    r, c = ct.shape
    V = np.sqrt(chi2 / (n * min(r - 1, c - 1)))
    return V, chi2, p

# Вычисление коэффициента для attack_type × severity
print("=" * 65)
print("КОЭФФИЦИЕНТ КРАМЕРА: attack_type × severity")
print("=" * 65)

ct = pd.crosstab(df['attack_type'], df['severity'])
V, chi2, p = cramers_v(ct)

print(f"\nχ² = {chi2:.4f}, n = {ct.values.sum()}, p = {p:.4e}")
print(f"V = {V:.4f}")

print(f"\nИнтерпретация:")
if V < 0.1:
    print(f"  V < 0.1 → пренебрежимо слабая связь")
elif V < 0.3:
    print(f"  0.1 ≤ V < 0.3 → слабая связь")
elif V < 0.5:
    print(f"  0.3 ≤ V < 0.5 → умеренная связь")
else:
    print(f"  V ≥ 0.5 → сильная связь")
```

```
================================================================
КОЭФФИЦИЕНТ КРАМЕРА: attack_type × severity
================================================================

χ² = 69.1353, n = 500, p = 4.6471e-10
V = 0.2147

Интерпретация:
  0.1 ≤ V < 0.3 → слабая связь
```

В библиотеке `scipy` для расчета коэффициента сопряжённости Крамера используется класс `association` с параметром `method = 'cramer'`.

In [52]:
```
association(ct, method = 'cramer')
```

Out[52]:
```
0.2146862346240983
```

Коэффициент взаимной сопряжённости Чупрова (Chuprov's T)

Коэффициент взаимной сопряженности Чупрова вычисляется по выражению:

$$T = \sqrt{\frac{\chi^2}{n \cdot \sqrt{(m-1) \cdot (k-1)}}}.$$

Свойства:

- Диапазон: [0, 1];
- $T = 0$: полная независимость ;
- $T = 1$: полная ассоциация (достижима только при $m = k$).

Рекомендуемые пороги для интерпретации:

- менее 0.1: пренебрежимо слабая связь;
- 0.1 – 0.3: слабая связь;
- 0.3 – 0.5: умеренная связь;
- более 0.5: сильная связь.

Оценим с помощью взаимной сопряжённости Чупрова зависимость признаков "attack_type" и "severity".

In [53]:
```
# Функция для расчёта коэффициента взаимной сопряжённости Чупрова
def chuprov_coefficient(ct):
    chi2, p, dof, exp = chi2_contingency(ct)
    n = ct.values.sum()
    r, c = ct.shape
    T = np.sqrt(chi2 / (n * np.sqrt((r - 1) * (c - 1))))
    return T, chi2, p

# Вычисление коэффициента для attack_type × severity
print("=" * 65)
print("КОЭФФИЦИЕНТ ЧУПРОВА: attack_type × severity")
print("=" * 65)

ct = pd.crosstab(df['attack_type'], df['severity'])
T, chi2, p = chuprov_coefficient(ct)
V, _, _ = cramers_v(ct)
```

```
print(f"χ² = {chi2:.4f}, n = {ct.values.sum()}")
print(f"T (Чупров) = {T:.4f}")

print(f"\nИнтерпретация:")
if T < 0.1:
    print(f"  T < 0.1 → пренебрежимо слабая связь")
elif T < 0.3:
    print(f"  0.1 ≤ T < 0.3 → слабая связь")
elif T < 0.5:
    print(f"  0.3 ≤ T < 0.5 → умеренная связь")
else:
    print(f"  T ≥ 0.5 → сильная связь")
```

```
============================================================
КОЭФФИЦИЕНТ ЧУПРОВА: attack_type × severity
============================================================
χ² = 69.1353, n = 500
T (Чупров) = 0.1998

Интерпретация:
  0.1 ≤ T < 0.3 → слабая связь
```

В библиотеке `scipy` для расчета коэффициента сопряжённости Чупрова используется класс `association` с параметром `method = 'tschuprow'`.

In [54]:
```
association(ct, method = 'tschuprow')
```

Out[54]:
```
0.1997880531235193
```

Рекомендации по выбору показателя оценивания зависимости категориальных признаков.

In [55]:
```
recommendations = pd.DataFrame({
    'Ситуация': [
        'Таблица 2×2, нужна быстрая оценка',
        'Таблица 2×2, малая выборка (n < 30)',
        'Таблица r×c, нужна нормированная мера [0,1]',
        'Таблица r×c, r ≠ c, нужна консервативная оценка',
        'Нужно проверить статистическую значимость',
        'Нужно направление связи (для 2×2)',
        'Признаки ординальные (есть порядок)',
    ],
    'Рекомендуемый метод': [
        "Коэффициент Юла (Q) / ф-коэффициент",
        "Точный критерий Фишера",
        "Коэффициент Крамера (V)",
        "Коэффициент Чупрова (T)",
        "Критерий хи-квадрат Пирсона",
        "Коэффициент Юла",
        "Ранговые корреляции (Спирмен, Кендалл)",
    ],
    'Пример в кибербезопасности': [
        'IOC × компрометация',
        'Пилотное тестирование нового правила IDS',
        'Тип атаки × целевая система',
        'Тип атаки (5) × сегмент сети (3)',
        'Любая пара категориальных признаков',
        'Срабатывание алерта × подтверждение атаки',
        'Уровень критичности × приоритет реагирования',
    ]
})
```

```
print("=" * 65)
print("РЕКОМЕНДАЦИИ ПО ВЫБОРУ МЕТОДА")
print("=" * 65)
for _, row in recommendations.iterrows():
    print(f"\n  Ситуация: {row['Ситуация']}")
    print(f"  Метод:    {row['Рекомендуемый метод']}")
    print(f"  Пример:   {row['Пример в кибербезопасности']}")
```

```
=================================================================
РЕКОМЕНДАЦИИ ПО ВЫБОРУ МЕТОДА
=================================================================

  Ситуация: Таблица 2×2, нужна быстрая оценка
  Метод:    Коэффициент Юла (Q) / ф-коэффициент
  Пример:   IOC × компрометация

  Ситуация: Таблица 2×2, малая выборка (n < 30)
  Метод:    Точный критерий Фишера
  Пример:   Пилотное тестирование нового правила IDS

  Ситуация: Таблица r×c, нужна нормированная мера [0,1]
  Метод:    Коэффициент Крамера (V)
  Пример:   Тип атаки × целевая система

  Ситуация: Таблица r×c, r ≠ c, нужна консервативная оценка
  Метод:    Коэффициент Чупрова (T)
  Пример:   Тип атаки (5) × сегмент сети (3)

  Ситуация: Нужно проверить статистическую значимость
  Метод:    Критерий хи-квадрат Пирсона
  Пример:   Любая пара категориальных признаков

  Ситуация: Нужно направление связи (для 2×2)
  Метод:    Коэффициент Юла
  Пример:   Срабатывание алерта × подтверждение атаки

  Ситуация: Признаки ординальные (есть порядок)
  Метод:    Ранговые корреляции (Спирмен, Кендалл)
  Пример:   Уровень критичности × приоритет реагирования
```

4. Корреляция между количественными и категориальными признаками

В задачах кибербезопасности часто возникает необходимость оценить, как категориальный признак (тип атаки, сегмент сети, уровень критичности) влияет на количественный показатель (время обнаружения, объём утечки, число затронутых хостов, интенсивность трафика).

Расширим датасет, добавив количественные признаки.

In [56]:
```
# @title
np.random.seed(42)

# ── Базовый датасет из Вопроса 3 ──────────────────────────────
n = 500

attack_type = np.random.choice(
    ['Phishing', 'DDoS', 'Malware', 'BruteForce', 'SQLi'],
    size=n, p=[0.30, 0.25, 0.20, 0.15, 0.10]
```

```
)

target_system = []
for a in attack_type:
    probs = {
        'Phishing':   [0.50, 0.30, 0.15, 0.05],
        'DDoS':       [0.10, 0.50, 0.35, 0.05],
        'Malware':    [0.40, 0.25, 0.15, 0.20],
        'BruteForce': [0.15, 0.60, 0.20, 0.05],
        'SQLi':       [0.10, 0.55, 0.30, 0.05],
    }[a]
    target_system.append(np.random.choice(
        ['Workstation', 'Server', 'Cloud', 'Mobile'], p=probs))

severity = []
for a in attack_type:
    probs = {
        'Phishing':   [0.25, 0.40, 0.25, 0.10],
        'DDoS':       [0.10, 0.40, 0.35, 0.15],
        'Malware':    [0.15, 0.30, 0.35, 0.20],
        'BruteForce': [0.30, 0.40, 0.25, 0.05],
        'SQLi':       [0.05, 0.20, 0.45, 0.30],
    }[a]
    severity.append(np.random.choice(
        ['Low', 'Medium', 'High', 'Critical'], p=probs))

time_period = np.random.choice(
    ['Morning', 'Afternoon', 'Evening', 'Night'],
    size=n, p=[0.20, 0.35, 0.30, 0.15])

# ── Количественные признаки с зависимостью от категориальных ──────

# 1. Интенсивность трафика (пакетов/с) — СИЛЬНАЯ зависимость от типа атаки
packet_rate = []
for a in attack_type:
    if a == 'DDoS':
        packet_rate.append(np.random.lognormal(mean=10, sigma=0.5))
    elif a == 'BruteForce':
        packet_rate.append(np.random.lognormal(mean=6, sigma=0.4))
    elif a == 'SQLi':
        packet_rate.append(np.random.lognormal(mean=5, sigma=0.5))
    elif a == 'Malware':
        packet_rate.append(np.random.lognormal(mean=4.5, sigma=0.6))
    else:  # Phishing
        packet_rate.append(np.random.lognormal(mean=3.5, sigma=0.7))
packet_rate = np.array(packet_rate)

# 2. Время обнаружения (минуты) — УМЕРЕННАЯ зависимость от критичности
detection_time = []
for s in severity:
    if s == 'Critical':
        detection_time.append(np.random.gamma(shape=2, scale=8))
    elif s == 'High':
        detection_time.append(np.random.gamma(shape=2.5, scale=12))
    elif s == 'Medium':
        detection_time.append(np.random.gamma(shape=3, scale=18))
    else:  # Low
        detection_time.append(np.random.gamma(shape=3.5, scale=25))
detection_time = np.array(detection_time)

# 3. Объём утечки данных (МБ) — ЗАВИСИМОСТЬ от типа атаки
data_exfil = []
for a in attack_type:
```

```
    if a == 'SQLi':
        data_exfil.append(np.random.lognormal(mean=5, sigma=1.0))
    elif a == 'Phishing':
        data_exfil.append(np.random.lognormal(mean=3.5, sigma=1.2))
    elif a == 'Malware':
        data_exfil.append(np.random.lognormal(mean=3, sigma=1.0))
    elif a == 'DDoS':
        data_exfil.append(np.random.lognormal(mean=1.5, sigma=0.8))
    else:  # BruteForce
        data_exfil.append(np.random.lognormal(mean=2, sigma=0.9))
data_exfil = np.array(data_exfil)

# 4. Число затронутых хостов — СИЛЬНАЯ зависимость от типа атаки
affected_hosts = []
for a in attack_type:
    if a == 'DDoS':
        affected_hosts.append(int(np.random.lognormal(mean=4, sigma=0.8)) + 1)
    elif a == 'Malware':
        affected_hosts.append(int(np.random.lognormal(mean=2.5, sigma=0.7)) + 1)
    elif a == 'Phishing':
        affected_hosts.append(int(np.random.lognormal(mean=2, sigma=0.9)) + 1)
    elif a == 'BruteForce':
        affected_hosts.append(int(np.random.lognormal(mean=1.5, sigma=0.5)) + 1)
    else:  # SQLi
        affected_hosts.append(int(np.random.lognormal(mean=2, sigma=0.6)) + 1)
affected_hosts = np.array(affected_hosts)

# 5. Бинарный признак: инцидент подтверждён (True/False)
# Зависимость от типа атаки
confirmed = []
for a in attack_type:
    p_confirm = {'SQLi': 0.85, 'DDoS': 0.90, 'Malware': 0.75,
                 'BruteForce': 0.65, 'Phishing': 0.55}[a]
    confirmed.append(np.random.random() < p_confirm)
confirmed = np.array(confirmed)

# ── Сборка DataFrame ──────────────────────────────────────────
df = pd.DataFrame({
    'attack_type':       attack_type,
    'target_system':     target_system,
    'severity':          severity,
    'time_period':       time_period,
    'packet_rate':       np.round(packet_rate, 1),
    'detection_time':    np.round(detection_time, 1),
    'data_exfil_mb':     np.round(data_exfil, 2),
    'affected_hosts':    affected_hosts,
    'incident_confirmed': confirmed,
})
```

In [57]:
```
df.head()
```

Out[57]:

	attack_type	target_system	severity	time_period	packet_rate	detection_time	data_exfil_n
0	DDoS	Cloud	Medium	Afternoon	14200.2	31.1	2.
1	SQLi	Server	High	Afternoon	98.2	21.0	851.
2	Malware	Workstation	Critical	Morning	78.6	9.0	51.
3	Malware	Mobile	High	Afternoon	112.2	44.7	15.
4	Phishing	Server	High	Afternoon	62.8	44.1	129.

Признаки:

- `attack_type` : тип атаки;
- `target_system` : целевая система;
- `severity` : уровень критичности инцидента;
- `time_period` : время суток;
- `packet_rate` : интенсивность трафика (пакетов/с);
- `detection_time` : время обнаружения (минуты);
- `data_exfil_mb` : объём утечки данных (МБ);
- `affected_hosts` : число затронутых хостов;
- `incident_confirmed` : инцидент подтверждён (True/False).

Визуальный анализ зависимости

Примеры использования в кибербезопасности.

- Анализ времени обнаружения по типам атак: аналитик SOC строит `boxplot` времени обнаружения для каждого типа атаки, чтобы выявить, какие атаки обнаруживаются дольше.
- Оценка ущерба по категориям: распределение объёма утечки данных по типам атак позволяет обосновать бюджет на защиту.
- Профилирование трафика: распределение интенсивности трафика по типам атак помогает калибровать пороги аномалий в NIDS.
- Сравнение эффективности защиты: время реагирования до и после внедрения SOAR-платформы.

Для визуального анализа влияния категориального признака на количественный признак рекомендуется использовать:

- сравнение гистограмм;
- сравнение графиков функций распределения (`ECDF`);
- сравнение графиков "ящик с усами" (`boxplot`).

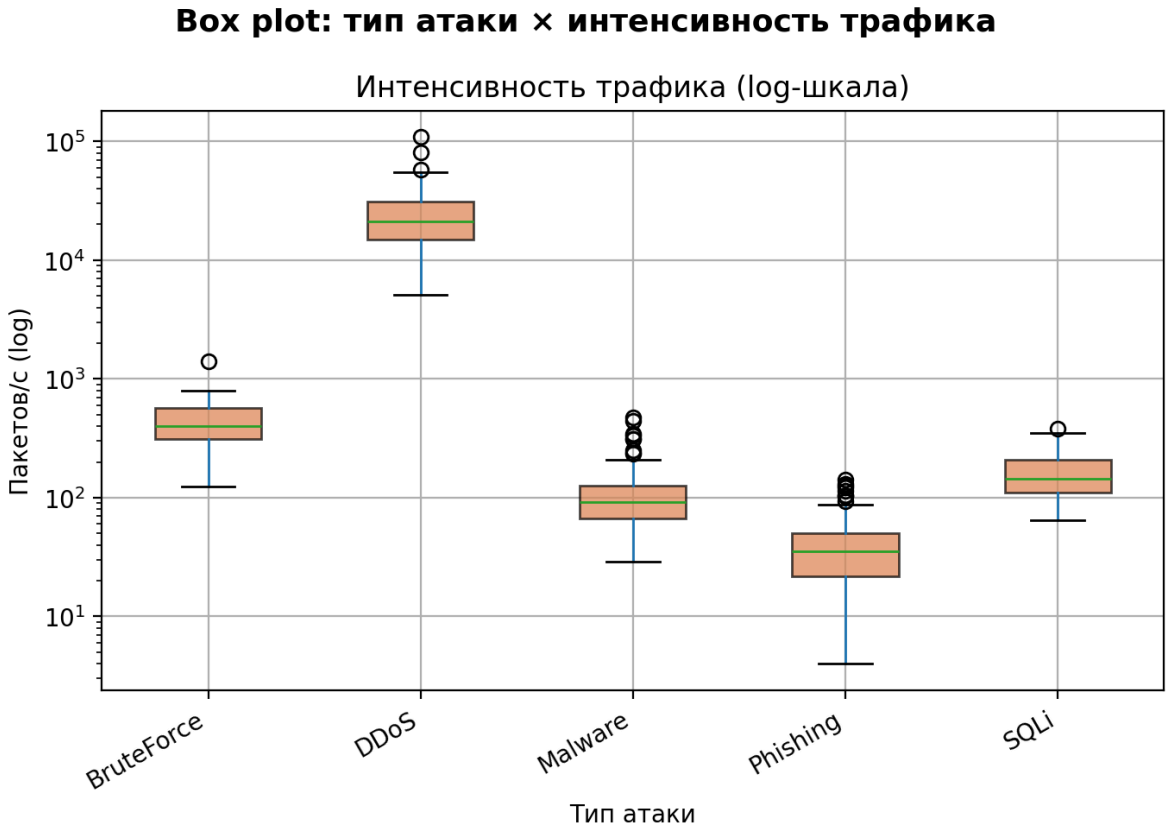
Ключевой принцип: если распределения количественного признака в разных категориях существенно различаются (разные медианы, разбросы, формы), это указывает на наличие связи.

Проведем визуальное оценивание влияния типа атаки ("attack_type") на интенсивность трафика ("packet_rate").

Сравнение диаграмм `boxplot`.

In [58]:
```
fig, axes = plt.subplots(figsize=(7, 5))
# С логарифмированием (для лучшей видимости)
df.boxplot(column='packet_rate', by='attack_type', ax=axes,
           patch_artist=True,
           boxprops=dict(facecolor='#DD8452', alpha=0.7))
axes.set_yscale('log')
axes.set_title('Интенсивность трафика (log-шкала)')
axes.set_xlabel('Тип атаки')
axes.set_ylabel('Пакетов/с (log)')
axes.set_xticklabels(axes.get_xticklabels(), rotation=30, ha='right')

plt.suptitle('Box plot: тип атаки × интенсивность трафика',
             fontsize=13, fontweight='bold')
plt.tight_layout()
```



Интерпретация `boxplot`: если медианы расположены на разных уровнях, а "ящики" не перекрываются — это признак различий между группами.

Сравнение гистограмм.

In [59]:
```
attack_types = ['Phishing', 'DDoS', 'Malware', 'BruteForce', 'SQLi']

# Общие параметры для сопоставимости
# Находим общий диапазон по всем данным
global_min = df['packet_rate'].min()
global_max = df['packet_rate'].max()
# Добавляем небольшой отступ, чтобы крайние значения не обрезались
padding = (global_max - global_min) * 0.05
```

```
bin_range = (global_min - padding, global_max + padding)

# Фиксируем число бинов
n_bins = 40

# Создаем сетку 2x3 (5 графиков + 1 пустая ячейка)
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes = axes.flatten()  # Превращаем в плоский массив для удобной индексации

colors = plt.cm.tab10(np.linspace(0, 1, len(attack_types)))

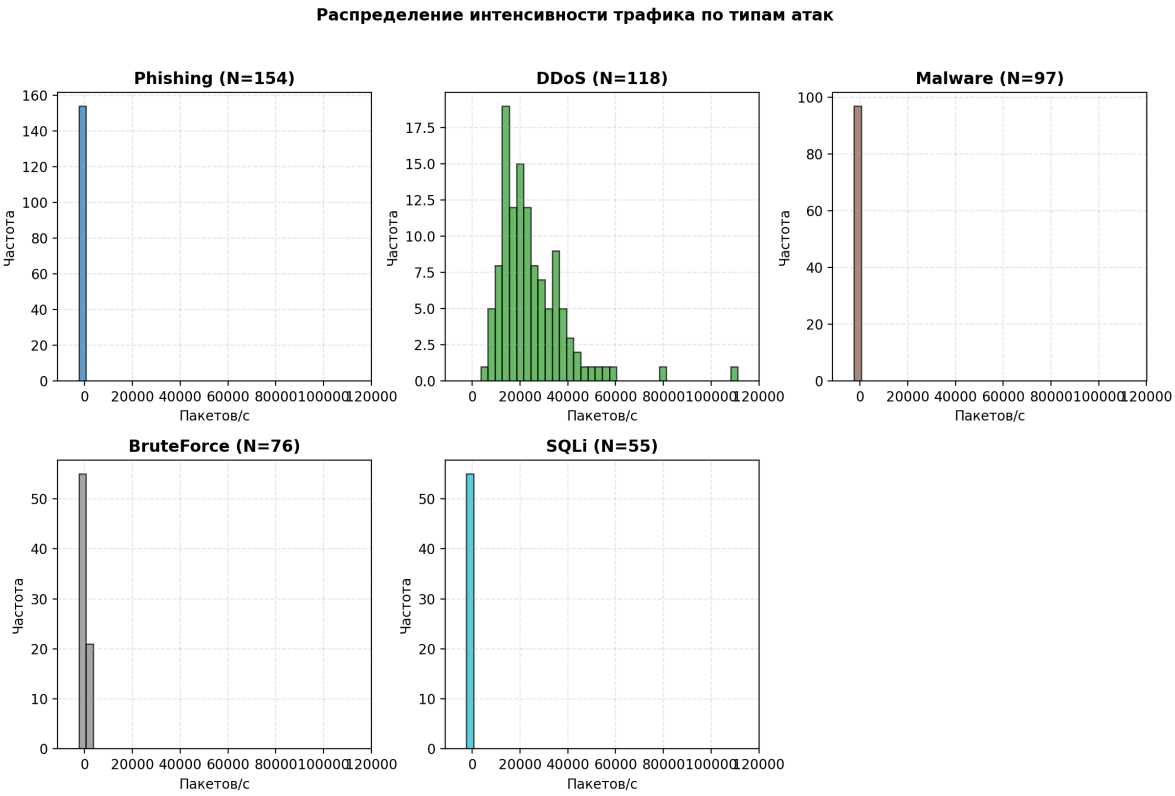
for i, atk in enumerate(attack_types):
    # Данные для текущего типа атаки
    data = df[df['attack_type'] == atk]['packet_rate']

    ax = axes[i]
    ax.hist(data, bins=n_bins, range=bin_range,
            color=colors[i], edgecolor='black', alpha=0.7)

    count = len(data)
    ax.set_title(f"{atk} (N={count})", fontsize=12, fontweight='bold')
    ax.set_xlabel("Пакетов/с", fontsize=10)
    ax.set_ylabel("Частота", fontsize=10)
    ax.grid(True, linestyle='--', alpha=0.3)

# Скрываем пустую ячейку (если есть)
axes[-1].axis('off')

plt.suptitle("Распределение интенсивности трафика по типам атак",
             fontsize=12, fontweight='bold', y=1.02)
plt.tight_layout()
```



Интерпретация гистограмм: если распределения количественного признака сильно отличаются, это указывает на наличие влияния на него категориального признака.

Поскольку форма гистограммы сильно зависит от количества интервалов, то целесообразно сравнивать графики функций распределений, построенных для каждой

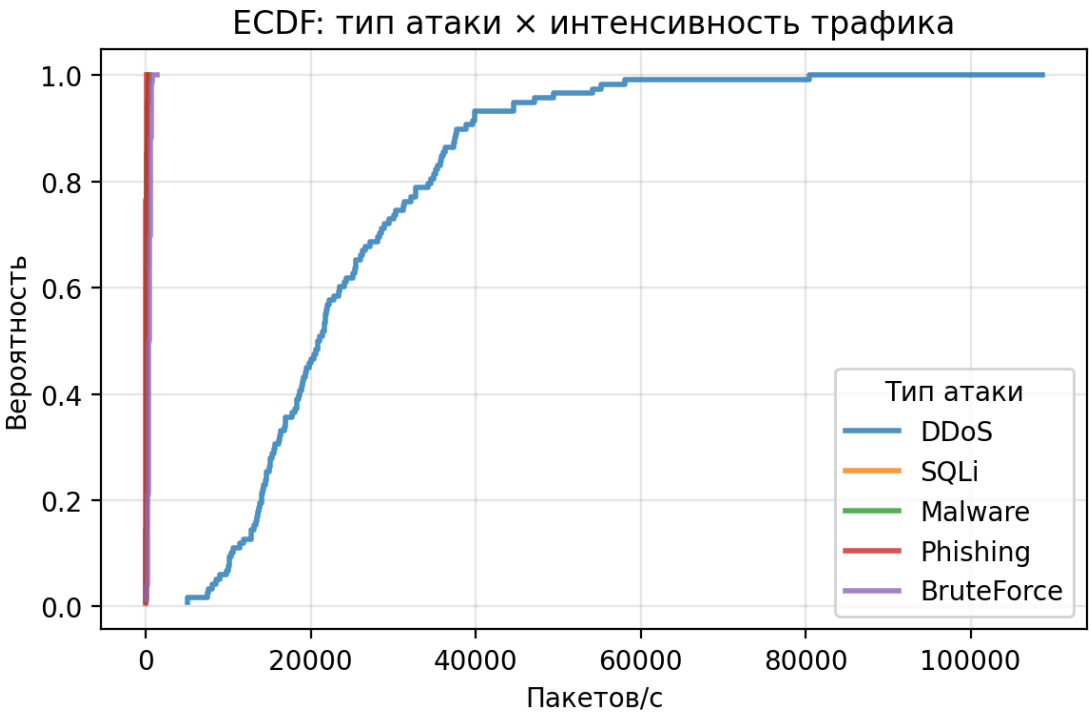
категории признака.

In [60]:
```
fig, ax = plt.subplots(figsize=(6, 4))

for at in df['attack_type'].unique():
    subset = np.sort(df[df['attack_type'] == at]['packet_rate'].values)
    ecdf_y = np.arange(1, len(subset) + 1) / len(subset)
    ax.step(subset, ecdf_y, label=at, linewidth=2, alpha=0.8)

ax.set_xlabel('Пакетов/с')
ax.set_ylabel('Вероятность')
ax.set_title('ECDF: тип атаки × интенсивность трафика')
ax.legend(title='Тип атаки')
ax.grid(alpha=0.3)

plt.tight_layout()
```



Интерпретация ECDF: если графики функций распределения количественного признака сильно расходятся для различных категорий категориального признака, это указывает на различия в распределениях и как следствие на наличие влияния на него категориального признака. Чем больше площадь между кривыми, тем сильнее различие.

Проверка гипотезы о независимости признаков

Визуальный анализ зависимости признаков формирует гипотезу. Для ее подтверждения нужны количественные метрики и статистические тесты.

Для выявления наличия зависимости между признаками проверяются статистические гипотезы о равенстве числовых характеристик количественного признака, вычисленных для различных категорий.

Основные критерии проверки равенства числовых характеристик:

- критерий Стьюдента (категориальный признак с 2-мя категориями);

- критерий Манна-Уитни (категориальный признак с 2-мя категориями);
- однофакторный дисперсионный анализ (ANOVA) (любое число категорий);
- критерий Краскела-Уоллиса (любое число категорий).

При использовании критериев Стьюдента и дисперсионного анализа должны выполняться жесткие требования:

- независимость наблюдений;
- приблизительная нормальность распределения внутри каждой группы;
- однородность дисперсий.

В задачах кибербезопасности эти условия, как правило, не выполняются. Поэтому на практике рекомендуется использовать непараметрические аналоги: критерий Манна-Уитни (бинарный категориальный признак) и критерий Краскела-Уоллиса (любой признак).

Критерий Манна-Уитни

Критерий Манна-Уитни (U-тест, критерий суммы рангов) — непараметрический аналог двухвыборочного t-теста. Сравнивает распределения (медианы) двух независимых категорий.

ВАЖНО! Критерий Манна-Уитни не проверяет гипотезу о равенстве медиан.

Критерий Манна-Уитни проверяет гипотезу о равенстве вероятностей того, что значения одного признака больше другого и наоборот:

Пусть есть две независимые выборки:

- X объёмом n_1,
- Y объёмом n_2.

Гипотезы:

- $H_0 : P(X > Y) = P(X < Y)$ - распределения в двух группах одинаковы;
- $H_1 : P(X > Y) \neq P(X < Y)$ - распределения различаются (или одна группа стохастически больше другой).

Пусть:

- n_1 и n_1 — размеры первой и второй выборок соответственно;
- R_1 и R_2 — суммы рангов в этих выборках (после объединения всех значений и присвоения рангов от 1 до $n_1 + n_2$).

Статистика критерия для каждой выборки:

$$U_1 = n_1 \cdot n_2 + \frac{n_1(n_1+1)}{2} - R_1, \quad U_2 = n_1 \cdot n_2 + \frac{n_2(n_2+1)}{2} - R_2.$$

При этом выполняется соотношение:

$$U_1 + U_2 = n_1 \cdot n_2.$$

U_1 — это число пар, где наблюдение из первой группы больше наблюдения из второй.

U_2 — то же самое, но для второй группы против первой.

Если первая группа систематически имеет бóльшие значения, то U_1 будет маленьким, а U_2 — большим (близким к $n_1 \cdot n_2$), и наоборот.

В качестве тестовой статистики обычно берут минимальное значение:

$$U = \min(U_1, U_2).$$

Для расчета критерия используется функция `mannwhitneyu` модуля `stats` библиотеки `scipy`.

Проверим гипотезу о равенстве распределений (медиан) признака `detection_time` (время обнаружения) для категорий признака `incident_confirmed` (инцидент подтверждён).

In [61]:
```
from scipy.stats import mannwhitneyu

print("=" * 65)
print("КРИТЕРИЙ МАННА-УИТНИ: incident_confirmed × detection_time")
print("=" * 65)

group_not_conf = df[df['incident_confirmed'] == False]['detection_time'].values
group_conf     = df[df['incident_confirmed'] == True]['detection_time'].values

# Двусторонний тест
U_stat, p_mw = mannwhitneyu(group_conf, group_not_conf, alternative='two-sided')

# Размер эффекта: r = 1 - 2U/(n1*n2)
r_effect = 1 - (2 * U_stat) / (len(group_conf) * len(group_not_conf))

print(f"\nРезультаты:")
print(f"  U = {U_stat:.1f}")
print(f"  p = {p_mw:.4e}")
print(f"  r (размер эффекта) = {r_effect:.4f}")

print(f"\nРешение (α = 0.05):")
if p_mw < 0.05:
    print(f"  p < 0.05 → распределения значимо различаются")
    if np.median(group_conf) < np.median(group_not_conf):
        print(f"  → Подтверждённые инциденты обнаруживаются БЫСТРЕЕ")
    else:
        print(f"  → Подтверждённые инциденты обнаруживаются МЕДЛЕННЕЕ")
else:
    print(f"  p ≥ 0.05 → значимых различий не обнаружено")
```

```
=================================================================
КРИТЕРИЙ МАННА-УИТНИ: incident_confirmed × detection_time
=================================================================

Результаты:
  U = 24119.0
  p = 6.0465e-01
  r (размер эффекта) = 0.0300

Решение (α = 0.05):
  p ≥ 0.05 → значимых различий не обнаружено
```

Таким образом, подтверждённые и неподтвержденные инциденты обнаруживаются примерно за одинаковое время.

Важное замечание. `scipy.stats.mannwhitneyu` вычисляет U для двух групп, но по умолчанию возвращает U для первой группы, а не минимум. Поэтому:

- если первая группа меньше второй — U будет близко к 0;
- если первая группа больше второй — U будет близко к $n_1 \cdot n_2$.

Как интерпретировать результаты критерия Манна–Уитни?

U**-статистика** — это число пар, в которых наблюдение из одной группы больше наблюдения из другой.
Малое значение U указывает на сильное разделение групп.

p**-value** — вероятность получить такое (или ещё более выраженное) разделение при условии, что распределения в группах одинаковы.

Интерпретация размера эффекта r (рангово-бисериальная корреляция):

- $|r| < 0{,}1$ — пренебрежимо малый эффект;
- $0{,}1 \leq |r| < 0{,}3$ — малый эффект;
- $0{,}3 \leq |r| < 0{,}5$ — средний эффект;
- $|r| \geq 0{,}5$ — большой эффект.

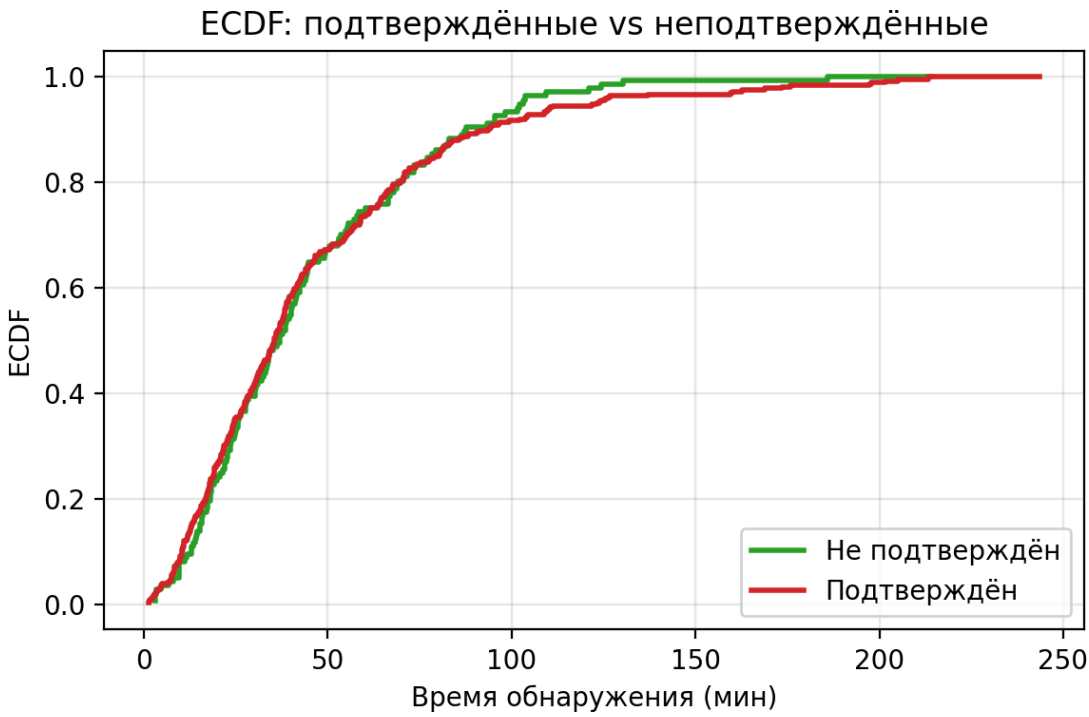
Визуальный анализ - сравнение функций распределения.

In [62]:
```
# ECDF для двух групп
fig, axes = plt.subplots(figsize=(6, 4))

for g, label, color in [(group_not_conf, 'Не подтверждён', '#2ca02c'),
                        (group_conf, 'Подтверждён', '#d62728')]:
    sorted_g = np.sort(g)
    ecdf_y = np.arange(1, len(sorted_g) + 1) / len(sorted_g)
    axes.step(sorted_g, ecdf_y, label=label, color=color, linewidth=2)

axes.set_xlabel('Время обнаружения (мин)')
axes.set_ylabel('ECDF')
axes.set_title('ECDF: подтверждённые vs неподтверждённые')
axes.legend()
axes.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



Критерий Краскела-Уоллиса

Критерий Краскела-Уоллиса — непараметрический аналог однофакторного ANOVA. Вместо сравнения средних сравнивает медианы (точнее, средние ранги) групп.

Гипотезы:

- H_0: распределения количественного признака во всех группах одинаковы.
- H_1: хотя бы одна группа отличается по положению (медиане).

Статистика критерия Краскела–Уоллиса:

$$H = \frac{12}{n(n+1)} \sum_{g=1}^{k} \frac{R_g^2}{n_g} - 3(n+1),$$

где:

- R_g — сумма рангов в группе g,
- n_g — размер группы g,
- n — общий объём выборки,
- k — число групп.

Статистика H асимптотически (при больших объёмах выборок) имеет распределение χ^2.

Алгоритм расчёта:

1. Все наблюдения объединяются и ранжируются (от наименьшего к наибольшему).
2. Ранги суммируются внутри каждой группы.
3. Вычисляется статистика H по формуле выше.
4. Полученное значение сравнивается с критическим значением $\chi^2_{\text{крит}}$ для уровня значимости α и $k-1$ степеней свободы (либо рассчитывается p-value).

5. Если $H > \chi^2_{\text{крит}}$ или p-value > α, нулевая гипотеза отклоняется - между категорями есть различия.

Проверим гипотезу о равенстве средних рангов признака `data_exfil_mb` (объём утечки данных) для различных категорий признака `attack_type` (тип атаки).

In [63]:
```
from scipy.stats import kruskal

print("=" * 65)
print("КРИТЕРИЙ КРАСКЕЛА-УОЛЛИСА: attack_type × data_exfil_mb")
print("=" * 65)

groups_kw = [df[df['attack_type'] == g]['data_exfil_mb'].values
             for g in df['attack_type'].unique()]
group_names = df['attack_type'].unique().tolist()

print(f"\nГипотезы:")
print(f"  H₀: распределения data_exfil_mb одинаковы для всех типов атак")
print(f"  H₁: хотя бы одно распределение отличается")

print(f"\nГрупповые медианы:")
for name, g in zip(group_names, groups_kw):
    print(f"  {name:12s}: n={len(g):3d}, median={np.median(g):10.2f} МБ")

# Критерий Краскела-Уоллиса
H_stat, p_kw = kruskal(*groups_kw)
print(f"\nРезультаты:")
print(f"  H = {H_stat:.2f}")
print(f"  p = {p_kw:.4e}")

print(f"\nРешение (α = 0.05):")
if p_kw < 0.05:
    print(f"  p < 0.05 → ОТВЕРГАЕМ H₀")
    print(f"  Распределения значимо различаются")
else:
    print(f"  p ≥ 0.05 → НЕ ОТВЕРГАЕМ H₀")
```

```
=================================================================
КРИТЕРИЙ КРАСКЕЛА-УОЛЛИСА: attack_type × data_exfil_mb
=================================================================

Гипотезы:
  H₀: распределения data_exfil_mb одинаковы для всех типов атак
  H₁: хотя бы одно распределение отличается

Групповые медианы:
  DDoS        : n=118, median=      4.15 МБ
  SQLi        : n= 55, median=    170.88 МБ
  Malware     : n= 97, median=     20.08 МБ
  Phishing    : n=154, median=     38.25 МБ
  BruteForce  : n= 76, median=      8.04 МБ

Результаты:
  H = 268.71
  p = 6.0657e-57

Решение (α = 0.05):
  p < 0.05 → ОТВЕРГАЕМ H₀
  Распределения значимо различаются
```

В случае отклонения нулевой гипотезы (хотя бы в одной категории распределение отличается от остальных) проводится Post-hoc анализ с использование критерия Манна-Уитни: определяется, какие именно пары категорий различаются. Обязательно с поправкой на множественные сравнения (Бонферрони, Холм).

In [64]:
```
from itertools import combinations

print("\n" + "=" * 65)
print("ПОПАРНЫЕ СРАВНЕНИЯ МАННА-УИТНИ С ПОПРАВКОЙ БОНФЕРРОНИ")
print("=" * 65)

alpha = 0.05
k = len(group_names)
n_comparisons = k * (k - 1) // 2
alpha_corrected = alpha / n_comparisons

print(f"\nЧисло попарных сравнений: {n_comparisons}")

posthoc_results = []
for (i, name_i), (j, name_j) in combinations(enumerate(group_names), 2):
    U, p_mw = mannwhitneyu(groups_kw[i], groups_kw[j], alternative='two-sided')
    significant = "✓" if p_mw < alpha_corrected else "✗"
    posthoc_results.append({
        'pair': f"{name_i} vs {name_j}",
        'U': U,
        'p_value': p_mw,
        'significant': significant
    })
    print(f"  {significant} {name_i:12s} vs {name_j:12s}: "
          f"U={U:8.0f}, p={p_mw:.4e}")

print(f"\n✓ — значимое различие после поправки Бонферрони")
```

```
=================================================================
ПОПАРНЫЕ СРАВНЕНИЯ МАННА-УИТНИ С ПОПРАВКОЙ БОНФЕРРОНИ
=================================================================

Число попарных сравнений: 10
  ✓ DDoS         vs SQLi        : U=      17, p=6.9000e-26
  ✓ DDoS         vs Malware     : U=    1500, p=1.3834e-20
  ✓ DDoS         vs Phishing    : U=    1251, p=3.7429e-34
  ✓ DDoS         vs BruteForce  : U=    2885, p=2.8219e-05
  ✓ SQLi         vs Malware     : U=    4944, p=2.6220e-18
  ✓ SQLi         vs Phishing    : U=    6994, p=7.7821e-13
  ✓ SQLi         vs BruteForce  : U=    4098, p=7.8203e-21
  ✓ Malware      vs Phishing    : U=    5168, p=4.0169e-05
  ✓ Malware      vs BruteForce  : U=    5295, p=8.6647e-07
  ✓ Phishing     vs BruteForce  : U=    9660, p=1.0533e-15

✓ — значимое различие после поправки Бонферрони
```

Ключевой вывод: все 10 пар демонстрируют статистически значимые различия ($p < 0,005$), что свидетельствует о том, что объём утечки данных существенно различается между всеми без исключения типами кибератак.

Интерпретация.

DDoS-атаки демонстрируют наиболее выраженную статистическую обособленность среди всех типов угроз. Особенно примечательно сравнение `DDoS` vs `Phishing` (

$p = 3,74 \cdot 10^{-34}$) — это наименьшее p-значение, указывающее на максимальную дистанцию между распределениями.

В контексте кибербезопасности это объяснимо:

- DDoS-атаки носят массовый, высокоинтенсивный и, как правило, кратковременный характер, направлены на нарушение доступности (нарушение триады CIA — Confidentiality, Integrity, Availability);
- Фишинг, напротив, является целенаправленной, низкоинтенсивной атакой на человеческий фактор, растянутой во времени и нацеленной на нарушение конфиденциальности.

SQL-инъекции статистически значимо отличаются от всех остальных типов атак. Это согласуется с их природой: `SQLi` — это эксплуатация уязвимостей на уровне приложений (OWASP Top 10), требующая специфического вектора атаки (HTTP-запросы к базам данных) и приводящая к нарушению целостности и конфиденциальности данных.

Для `Malware` vs. `Phishing` и `BruteForce` различия значимы, но p-values здесь на порядки выше, чем для пар с участием `DDoS`. Это может указывать на то, что вредоносное ПО и фишинг/брутфорс имеют частично пересекающиеся профили по исследуемому показателю. Действительно, в современных APT-кампаниях malware часто выступает полезной нагрузкой, доставляемой через фишинговые письма, а брутфорс может использоваться для латерального перемещения уже заражённой системы.

Для пары `Phishing` и `BruteForce` наибольшее значение U (9 660) при крайне малом p-value говорит о том, что распределения этих двух типов атак перекрываются в наибольшей степени, однако различие всё равно статистически надёжно. С позиции кибербезопасности это логично: обе атаки направлены на компрометацию учётных данных, но различаются механизмом (социальная инженерия vs. автоматизированный перебор).

Полученные результаты статистически обосновывают необходимость дифференцированного, типоспецифичного подхода к проектированию защитных механизмов, правил обнаружения и процедур реагирования на инциденты информационной безопасности.

Показатели зависимости

Точечно-бисериальный коэффициент корреляции

Если анализируется зависимость между одним количественным признаком и одним категориальным, то можно вычислить следующие показатели корреляционной зависимости:

- точечно-бисериальный коэффициент корреляции, в случае, если категориальный признак имеет дихотомическую шкалу;
- корреляционное отношение в случае множества категорий.

Для расчета точечно-бисериального коэффициента корреляции используется выражение вида:

$$r_{pb} = \frac{\bar{x}_1 - \bar{x}_0}{s} \sqrt{\frac{n_1 n_0}{n \cdot (n-1)}},$$

где:

- $\bar{x}_1$ и $\bar{x}_0$ - средние значения количественного признака для каждой категорий независимого признака;
- s - стандартное отклонение зависимого количественного признака;
- n_1, n_0 - объемы выборочных совокупностей в каждой категории.

Свойства:

- Диапазон значений: $[-1, +1]$.
- Интерпретация знака:
  - $r_{pb} = 0$: средние в группах равны;
  - положительный r_{pb} означает, что группа с меткой 1 имеет более высокое среднее значение количественного признака по сравнению с группой 0;
  - отрицательный r_{pb} — наоборот: у группы 1 среднее ниже.
- Интерпретации силы связи:

$\lvert r_{pb} \rvert$	**Интерпретация**
$< 0{,}1$	Пренебрежимо слабая связь
$0{,}1 – 0{,}3$	Слабая связь
$0{,}3 – 0{,}5$	Умеренная связь
$> 0{,}5$	Сильная связь

- **p-value** проверяет нулевую гипотезу: $H_0 : r_{pb} = 0 \quad (\text{средние в группах равны})$. Если $p < 0{,}05$, нулевая гипотеза отвергается — связь статистически значима.

Важное свойство: r_{pb} тождественно равен корреляции Пирсона между бинарным и непрерывным признаком.

Оценим наличие зависимости между признаками `incident_confirmed` (подтверждение инцидента) и `detection_time` (время выявления инцидента).

Для расчета коэффициента можно использовать функцию `pointbiserialr()` модуля `stats` библиотеки `SciPy`.

Если категориальный признак представлен в номинальной или порядковой шкале, его предварительно необходимо закодировать числовыми значениями, например, 0 и 1.

In [65]:
```
print("=" * 65)
print("ТОЧЕЧНАЯ БИСЕРИАЛЬНАЯ КОРРЕЛЯЦИЯ")
print("=" * 65)

var_binary = 'incident_confirmed'
var_quant  = 'detection_time'

group_0 = df[df[var_binary] == False][var_quant].values
group_1 = df[df[var_binary] == True][var_quant].values

print(f"\nГруппа 0 ({var_binary}=False): n={len(group_0)}, "
```

```
      f"mean={group_0.mean():.2f}, std={group_0.std():.2f}")
print(f"Группа 1 ({var_binary}=True):  n={len(group_1)}, "
      f"mean={group_1.mean():.2f}, std={group_1.std():.2f}")

# Расчёт через scipy
r_pb, p_pb = stats.pointbiserialr(df[var_binary].astype(int), df[var_quant])

# Расчёт вручную (для проверки)
X = df[var_quant].values
y = df[var_binary].astype(int).values
mean_1 = X[y == 1].mean()
mean_0 = X[y == 0].mean()
S_X = X.std(ddof=1)
n_1, n_0 = y.sum(), len(y) - y.sum()
r_pb_manual = (mean_1 - mean_0) / S_X * np.sqrt(n_0 * n_1 / (len(y) * (len(y)-1)))

# Проверка через корреляцию Пирсона
r_pearson = np.corrcoef(y, X)[0, 1]

print(f"\nРезультаты:")
print(f"  r_pb (scipy)   = {r_pb:.4f}, p = {p_pb:.4e}")
print(f"  r_pb (вручную) = {r_pb_manual:.4f}")
print(f"  r_pearson      = {r_pearson:.4f}")

print(f"\nИнтерпретация:")
if abs(r_pb) < 0.1:
    strength = "пренебрежимо слабая"
elif abs(r_pb) < 0.3:
    strength = "слабая"
elif abs(r_pb) < 0.5:
    strength = "умеренная"
else:
    strength = "сильная"
direction = "положительная" if r_pb > 0 else "отрицательная"
print(f"  Связь: {strength}, {direction}")
if (r_pb < 0) & (r_pb > 0.3):
    print(f"  → Подтверждённые инциденты обнаруживаются БЫСТРЕЕ")
elif (r_pb < 0) & (r_pb < -0.3):
    print(f"  → Подтверждённые инциденты обнаруживаются МЕДЛЕННЕЕ")
else:
    print(f'→ Зависимости между признаками нет')
```

```
============================================================
ТОЧЕЧНАЯ БИСЕРИАЛЬНАЯ КОРРЕЛЯЦИЯ
============================================================

Группа 0 (incident_confirmed=False): n=137, mean=45.75, std=34.38
Группа 1 (incident_confirmed=True):  n=363, mean=46.53, std=39.96

Результаты:
  r_pb (scipy)   = 0.0090, p = 8.4029e-01
  r_pb (вручную) = 0.0090
  r_pearson      = 0.0090

Интерпретация:
  Связь: пренебрежимо слабая, положительная
→ Зависимости между признаками нет
```

Эта-коэффициент (корреляционное отношение)

Эта-коэффициент (η) измеряет связь между категориальным признаком с любым числом категорий и количественным признаком.

Для расчета корреляционного отношения используются следующие выражения:

- взвешенные по количеству элементов квадраты отклонений общего среднего от внутригрупповых средних:

$$ss_{\text{межгр}} = \sum_{i=1}^{m} n_j \cdot (\overline{y}_j - \overline{y})^2$$

- общая сумма квадратов:

$$ss_{\text{общ}} = \sum_{i=1}^{n} (y_i - \overline{y})^2,$$

m - количество уникальных значений категориального признака;

n_j - количество объектов, для которых категориальный признака имеет j-е значение.

Корреляционное отношение вычисляется как отношение вида:

$$\eta^2 = \sqrt{\frac{ss_{\text{межгр}}}{ss_{\text{общ}}}}.$$

Свойства:

- Диапазон значений: $[0, 1]$.
- Интерпретация крайних значений:
  - $\eta = 0$: средние во всех группах равны, групповая принадлежность не объясняет разброс количественного признака.
  - $\eta = 1$: вся дисперсия количественного признака объяснена групповой принадлежностью (внутри групп нет разброса).
- Квадрат корреляционного отношения η^2 интерпретируется как **доля дисперсии количественного признака, объяснённая категориальным признаком.

In [66]:
```python
# Функция для расчета корреляционного отношения
def eta_coefficient(df, cat_var, quant_var):
    groups = [df[df[cat_var] == g][quant_var].values
              for g in df[cat_var].unique()]

    grand_mean = df[quant_var].mean()

    # Суммы квадратов
    ss_total = sum((x - grand_mean)**2 for x in df[quant_var])
    ss_between = sum(len(g) * (g.mean() - grand_mean)**2 for g in groups)
    ss_within = sum(((g - g.mean())**2).sum() for g in groups)

    eta_sq = ss_between / ss_total
    eta = np.sqrt(eta_sq)

    # F-статистика
    k = len(groups)
    n_total = len(df)
    ms_between = ss_between / (k - 1)
    ms_within = ss_within / (n_total - k)
    F = ms_between / ms_within
    p_value = 1 - stats.f.cdf(F, k - 1, n_total - k)

    return eta, eta_sq, F, p_value, groups
```

Оценим степень зависимости признаков `attack_type` (тип атаки) и `packet_rate` (интенсивность трафика).

In [67]:
```python
print("=" * 65)
print("ЭТА-КОЭФФИЦИЕНТ: attack_type × packet_rate")
print("=" * 65)

eta, eta_sq, F, p, groups = eta_coefficient(df, 'attack_type', 'packet_rate')

print(f"\nГрупповые средние:")
for i, g_name in enumerate(df['attack_type'].unique()):
    g = groups[i]
    print(f"  {g_name:12s}: n={len(g):3d}, mean={g.mean():10.1f}, "
          f"std={g.std():10.1f}")

print(f"\nРезультаты:")
print(f"  η  = {eta:.4f}")
print(f"  η² = {eta_sq:.4f}  → {eta_sq*100:.1f}% дисперсии packet_rate "
      f"объяснено типом атаки")
print(f"  F  = {F:.2f}")
print(f"  p  = {p:.4e}")

print(f"\nИнтерпретация:")
if eta < 0.1:
    print(f"  η < 0.1 → связь пренебрежимо слабая")
elif eta < 0.3:
    print(f"  0.1 ≤ η < 0.3 → слабая связь")
elif eta < 0.5:
    print(f"  0.3 ≤ η < 0.5 → умеренная связь")
else:
    print(f"  η ≥ 0.5 → сильная связь")
```

```
================================================================
ЭТА-КОЭФФИЦИЕНТ: attack_type × packet_rate
================================================================

Групповые средние:
  DDoS        : n=118, mean=   24667.9, std=   14457.6
  SQLi        : n= 55, mean=     162.6, std=      70.9
  Malware     : n= 97, mean=     115.9, std=      81.0
  Phishing    : n=154, mean=      41.6, std=      27.1
  BruteForce  : n= 76, mean=     435.2, std=     190.2

Результаты:
  η  = 0.8289
  η² = 0.6871  → 68.7% дисперсии packet_rate объяснено типом атаки
  F  = 271.76
  p  = 1.1102e-16

Интерпретация:
  η ≥ 0.5 → сильная связь
```

Таким образом, тип атаки существенно влияет на интенсивность трафика.

Литература

1. Титов А.Н. Решение задач теории вероятностей и математической статистики в Python : учеб.-метод. пособие / Р.Ф. Тазиева; Казан. нац. исслед. технол. ун-т; А.Н. Титов. — Казань : КНИТУ, 2022. — 144 с.
2. Груздев А. В., Хейдт М. Изучаем Pandas. 2-ое изд., испр. и доп. - Москва: ДМК Пресс, 2019. - 1700 с.
3. Протодьяконов А.В. Алгоритмы Data Science и их практическая реализация на Python : учебное пособие / А.В. Протодьяконов, П.А. Пылов, В.Е. Садовников. - Москва : Инфра-Инженерия, 2022. - 392 с.
4. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2022 — Часть 1 — 2022. — 107 с.
5. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2023 — Часть 2 — 2023. — 92 с.